

L'editor di rsedit

Architettura della libreria `rsedit_core` e del front-end TUI — manuale per lo sviluppatore

Indice

1	Introduzione	3
1.1	Come leggere questo manuale	3
1.2	Comportamento e difetti	3
1.3	Convenzioni	3
2	Panoramica	4
2.1	Gli strati	4
2.2	Dove sta cosa	4
2.3	La superficie pubblica	5
2.4	I thread	5
3	Il ciclo di vita del programma	7
3.1	Dal main al ciclo degli eventi	7
3.2	Il ciclo degli eventi della TUI	8
4	Lo stato: EditorState e i compartimenti	10
4.1	La facciata	10
4.2	Le regole dei compartimenti	10
4.3	L'ordine dei lock	11
4.4	Accesso ai buffer	11
4.5	L'editor come contesto dell'interprete	12
5	Dal tasto al comando	13
5.1	Il vocabolario dei tasti	13
5.2	<code>handle_key_event</code>	13
5.3	L'argomento prefisso	14
5.4	La risoluzione: <code>resolve_key_sequence</code>	14
5.5	<code>run_command_form</code> : l'esecuzione di un comando	15
5.6	La cattura di una sequenza	17
6	Comandi e argomenti	18
6.1	Che cos'è un comando	18
6.2	Le specifiche degli argomenti	18
6.3	<code>call-interactively</code> e la catena degli argomenti	18
7	Minibuffer e ricerca incrementale	20
7.1	Il minibuffer	20
7.2	La ricerca incrementale	20
8	I buffer	21
8.1	La struttura Buffer	21
8.2	BufferTrait e GapBuffer	21
8.3	Le due porte	22
8.4	Undo	23
8.5	Mark, regione, overlay e testo virtuale	25
8.6	Le cache del testo	25

8.7	File su disco	26
9	Il testo: ricerca, sostituzione, kill ring, rettangoli e liste di risultati	27
9.1	Posizioni: sempre caratteri	27
9.2	Pattern: che cosa cercare	28
9.3	La sessione di ricerca incrementale: Isearch	29
9.4	La sessione di sostituzione: Replace	30
9.5	Il kill ring	32
9.6	I rettangoli	33
9.7	Le liste di risultati	33
10	Modi	35
10.1	Il modo maggiore	35
10.2	Due lessici sullo stesso testo	35
10.3	Lo scanner delle espressioni	36
10.4	Facce e temi	38
11	Il lavoro in background	39
11.1	La regola	39
11.2	Lo scheduler	39
11.3	Un turno dell'evidenziatore	40
11.4	I comandi di shell	41
11.5	Worker e lavori in Lisp	41
11.6	Le callback dovute	41
11.7	Quando ridisegnare	42
12	Lo schermo	43
12.1	Finestre	43
12.2	snapshot: dalla memoria alla fotografia	43
12.3	Il renderer della TUI	44
12.4	Il mouse	44
13	Macro di tastiera	45
14	Le primitive dell'editor	46
14.1	Il registro	46
15	I moduli Lisp	48
16	I test	50
17	Ricette	51
17.1	Una primitiva o un comando in Rust	51
17.2	Un comando in Lisp	51
17.3	Un modo maggiore	51
17.4	Un lavoro di sfondo	51
17.5	Un modulo Lisp nuovo	51
17.6	Un nuovo front-end	51
18	Appendice	53
18.1	Glossario	53
18.2	Mantenere aggiornato questo manuale	53
18.3	Elenco delle primitive dell'editor	53

1 Introduzione

Questo manuale descrive la libreria `rsedit_core` (`core/src/`, esclusa la cartella `lisp/`) e il front-end a terminale che la usa (`src/`). È il compagno di *L'interprete Lisp di rsedit* (`architettura-interprete.typ`), che descrive il linguaggio: qui l'interprete è dato per noto, e si spiega come l'editor lo ospita, come trasforma un tasto in un comando, come conserva il testo, come lavora in background e come descrive lo schermo a chi lo disegna.

La libreria è un editor *senza testa*. Non apre un terminale, non legge la tastiera e non disegna: riceve eventi già tradotti nel proprio vocabolario (`KeyEvent`, `MouseEvent`, testo incollato, dimensioni) e restituisce, quando le viene chiesto, una fotografia immutabile di ciò che andrebbe mostrato (`FrameSnapshot`). Tutto ciò che sta in mezzo — buffer, finestre, modi, comandi, lavori di sfondo — è in `rsedit_core`, e quasi tutto il comportamento visibile all'utente è scritto in Lisp nei moduli di `core/lisp/`.

Due confini reggono l'architettura, e sono verificati dai test (`tests::layering_tests`): `lisp/` non nomina alcun tipo dell'editor, e `ui/` non nomina nulla di `editor/`. Il primo permette di provare e riusare l'interprete da solo; il secondo permette di scrivere un renderer contro una fotografia invece che contro gli interni della libreria.

1.1 Come leggere questo manuale

Il Sezione 2 e il Sezione 3 danno il quadro: gli strati, i thread, che cosa succede dall'avvio al ciclo degli eventi. Poi il manuale segue il percorso di un tasto: lo stato che attraversa (Sezione 4), la risoluzione e l'esecuzione del comando (Sezione 5, Sezione 6, Sezione 7), il testo che il comando modifica (Sezione 8, Sezione 9), i modi che lo interpretano (Sezione 10), il lavoro che continua in background (Sezione 11) e la fotografia che il front-end disegna (Sezione 12). Gli ultimi capitoli sono di riferimento.

Gli estratti di codice sono presi dal sorgente e semplificati: commenti tolti, parametri generici abbreviati, rami ripetitivi riassunti. Ogni riga di un estratto corrisponde a una riga del file indicato sopra di esso; il percorso è relativo a `core/src/`.

Ogni affermazione è stata letta nel codice; i comportamenti che dipendono dall'esecuzione (avvio, undo, comandi di shell, colorazione) sono stati osservati con piccoli programmi che usano la libreria come farebbe un front-end.

1.2 Comportamento e difetti

Questo manuale descrive il comportamento *attuale* del codice, anche dove quel comportamento è un difetto. I difetti sono descritti, con la riproduzione e una correzione proposta, nel *Rapporto sui problemi del codice* (`rapporto-problemi.typ`). Dove il manuale descrive un punto che il rapporto considera un difetto, un rimando come (`→ BUF-1`) indica la scheda corrispondente; quando il difetto viene corretto si aggiorna la descrizione qui e si toglie la scheda là.

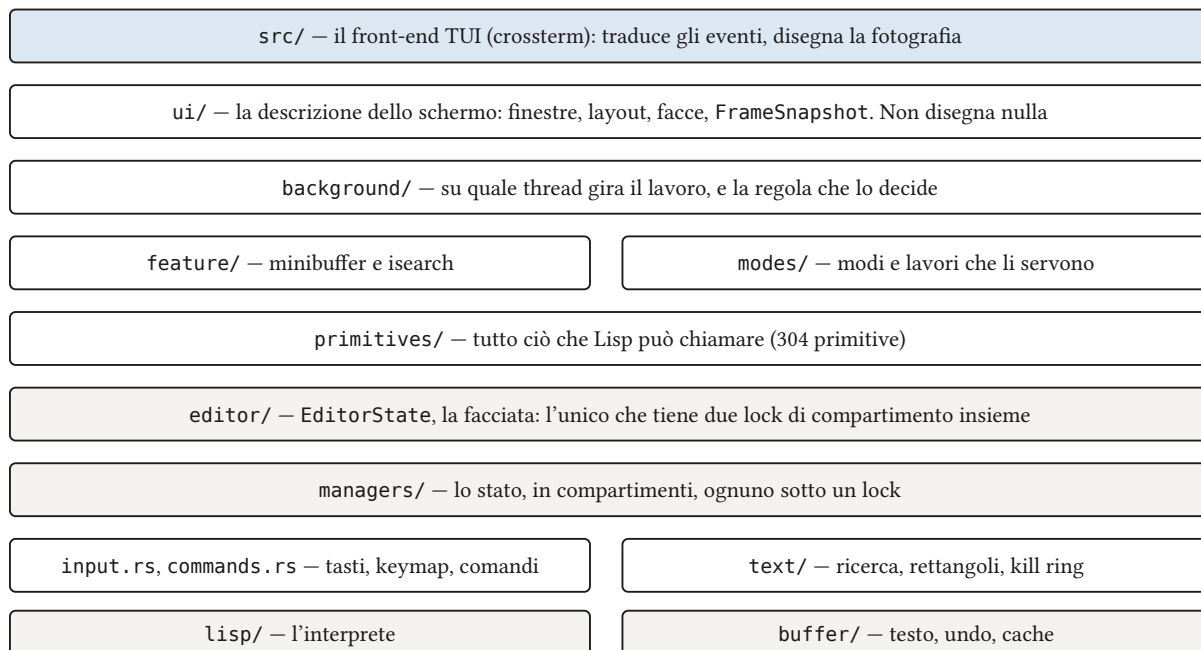
1.3 Convenzioni

- I nomi in `monospazio` sono identificatori: tipi e funzioni Rust (`run_command_form`), primitive e variabili Lisp (`call-interactively`, `mouse-mode`), file (`editor/keys.rs`, relativo a `core/src/` salvo indicazione diversa).
- Nei diagrammi le caselle grigie sono dati o stato, quelle bianche funzioni o passi, quelle azzurre appartengono a un altro thread o al front-end, quelle rosate sono errori. Nei diagrammi di sequenza le frecce continue sono chiamate e quelle tratteggiate risposte.
- «Primitiva» è una funzione Rust registrata nell'ambiente Lisp; «comando» è una funzione (Rust o Lisp) registrata anche nel registro dei comandi, e quindi raggiungibile con `M-x` e capace di farsi chiedere gli argomenti.

2 Panoramica

2.1 Gli strati

`lib.rs` elenca i moduli in ordine di dipendenza: ognuno usa quelli sopra di sé e mai quelli sotto. La figura li mostra dal basso verso l'alto, cioè nella direzione in cui si costruisce.



Le frecce sono omesse perché la regola è una sola: ogni livello può usare tutti quelli disegnati sotto di lui. `lisp/` e `buffer/` non si conoscono; si incontrano solo in `editor/`.

2.2 Dove sta cosa

Percorso	Contenuto
<code>lib.rs</code>	Ordine dei moduli e superficie pubblica (<code>pub use</code>). Tutto il resto è <code>pub(crate)</code> .
<code>buffer/</code>	<code>Buffer</code> , <code>BufferTrait</code> , <code>GapBuffer</code> , <code>undo</code> , <code>mark</code> , <code>overlay</code> , testo virtuale, cache di colorazione (<code>syntax.rs</code>) e di scansione (<code>scan.rs</code>), timbro del file su disco (<code>disk.rs</code>).
<code>text/</code>	Ciò che si può calcolare sul testo conoscendo solo <code>BufferTrait</code> : ricerca e isearch, rettangoli, kill ring, liste di posizioni (<code>results.rs</code>).
<code>input.rs</code>	<code>KeyEvent</code> , <code>KeyCode</code> , <code>MouseEvent</code> , <code>Keymap</code> , <code>TransientKeymap</code> , il parser dei nomi dei tasti, le keymap predefinite.
<code>commands.rs</code>	<code>ArgSpec</code> , <code>PrefixArg</code> , <code>Invocation</code> , <code>PendingCommand</code> : il vocabolario dei comandi.
<code>managers/</code>	I compartimenti dello stato: <code>Buffers</code> , <code>Windows</code> , <code>Modes</code> , <code>Commands</code> , <code>KillYank</code> , <code>Macros</code> , <code>History</code> , <code>Log</code> , <code>Runtime</code> .
<code>editor/</code>	<code>EditorState</code> e i suoi metodi, divisi per compartimento (<code>buffers.rs</code> , <code>windows.rs</code> ...) o per preoccupazione (<code>keys.rs</code> , <code>mouse.rs</code> , <code>frame.rs</code> , <code>boot.rs</code> , <code>settings.rs</code> , <code>background.rs</code> , <code>macros.rs</code>).
<code>primitives/</code>	Le 304 primitive dell'editor, in 28 moduli, e <code>install_primitives</code> .
<code>modes/</code>	<code>MajorMode</code> , grammatica di colorazione (<code>syntax.rs</code>), tabella sintattica e scanner di espressioni (<code>sexp.rs</code>), i lavori di sfondo <code>Highlighter</code> , <code>Prescanner</code> , <code>FileWatcher</code> , <code>AutoSaver</code> .

Percorso	Contenuto
feature/	Le due funzionalità che non possono dipendere da un file Lisp: il minibuffer e la ricerca incrementale.
background/	Lo scheduler, <code>ScheduledTask</code> , <code>ImmediateTask</code> , i worker Lisp e <code>background-call</code> .
ui/	Albero delle finestre, finestre flottanti, layout delle righe, facce e temi, <code>FrameSnapshot</code> .
tests/	Circa ottanta file di test dell'editor, più <code>perf/</code> .
../lisp/	I moduli Lisp distribuiti con l'editor (venti file).
../build.rs	Copia i moduli Lisp in <code>target/<profilo>/data/lisp</code> , accanto all'eseguibile.
../../src/	Il front-end: <code>main.rs</code> (avvio) e <code>tui.rs</code> (ciclo degli eventi e disegno).

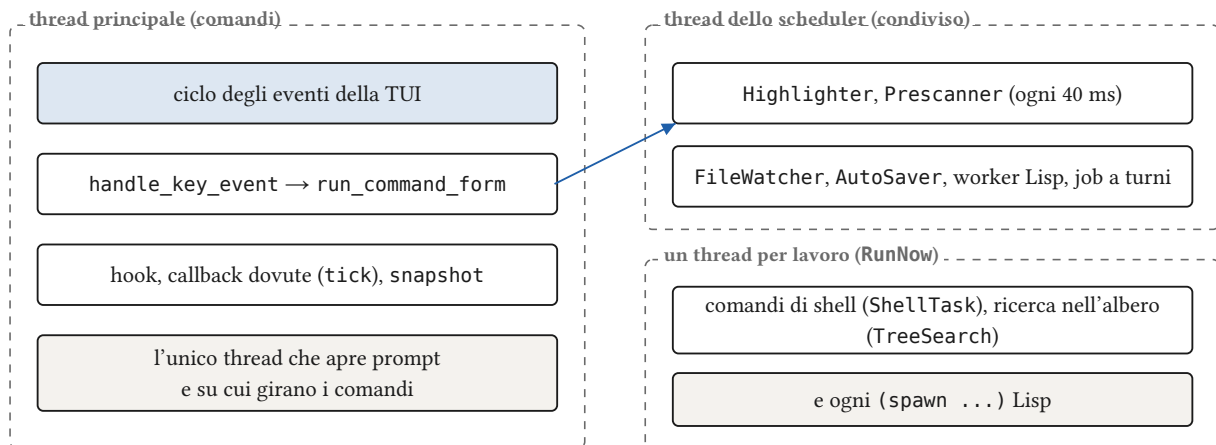
2.3 La superficie pubblica

Un front-end vede soltanto ciò che `lib.rs` riesporta: `EditorState`, `create_global_env`, `BufferTrait`, `GapBuffer`, i tipi di input (`KeyEvent`, `KeyCode`, `KeyModifiers`, `MouseEvent`, `MouseKind`, `MouseButton`), i tipi della fotografia (`FrameSnapshot`, `RenderableWindowView`, `Highlight`, `GutterCell`, `Separator`, `Rect`, `Face`, `Style`, `Color`, `Theme`, `NAMED_COLORS`), dall'interprete `Env`, `LispContext`, `Parser` ed `eval`, più `mouse_mode`, `MOUSE_MODE`, `CONFIG_DIR`, `XDG_CONFIG_HOME` e l'alias `ELispExp<B>` (`LispExp<EditorState<B>>`). I metodi di `EditorState` che un front-end usa sono pochi:

Metodo	Uso
<code>create_global_env::()</code>	Costruisce editor e ambiente Lisp, carica la configurazione.
<code>handle_key_event(ev, &env)</code>	Un tasto premuto.
<code>handle_paste(text, &env)</code>	Un incolla «bracketed», come un solo comando.
<code>handle_mouse_event(ev, &env)</code>	Un evento del mouse; risponde se ha cambiato qualcosa.
<code>resize(env, w, h)</code>	Nuove dimensioni del terminale.
<code>snapshot(&env, w, h)</code>	La fotografia da disegnare.
<code>next_redraw_in(&env, , &frame)</code> <code>drag_scroll_in()</code>	Quanto si può aspettare prima di ridisegnare anche senza eventi.
<code>tick(&env)</code>	Il tempo è scaduto: esegue ciò che era in attesa (callback di sfondo, scorrimento durante il trascinamento).
<code>is_running()</code>	Falso dopo <code>quit</code> .
<code>enable_log_file</code> , <code>log_diagnostic</code> , <code>set_echo_message</code>	Diagnostica.

2.4 I thread

L'editor usa più thread, ma uno solo esegue i comandi. La figura mostra chi esiste e che cosa condividono.

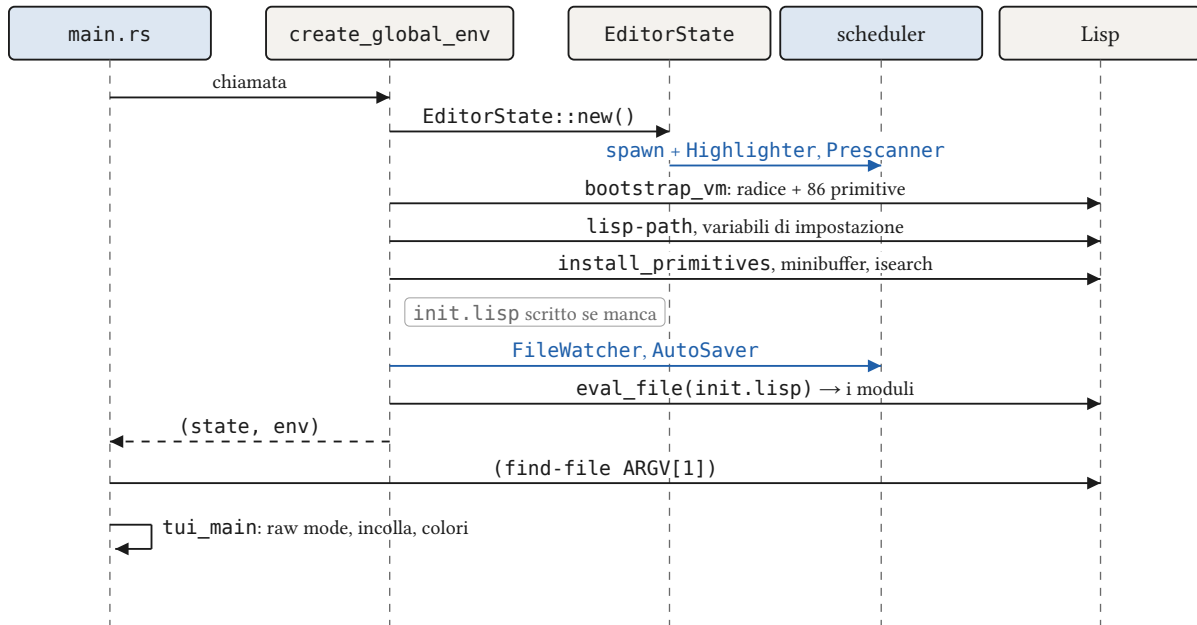


La freccia è la mailbox (`worker_mailbox`) con cui il thread dei comandi consegna lavoro allo scheduler. Tutti condividono lo stesso `EditorState` (clonato: ogni campo è un `Arc`) e lo stesso ambiente Lisp. Il carburante, il permesso di `yield`, il contrassegno del comando per l'undo e i dati di `string-match` sono invece per thread (variabili `thread_local!`). I lavori di sfondo non chiamano mai Lisp sul proprio thread quando l'effetto deve toccare l'interfaccia: lasciano una *callback dovuta*, che il thread principale esegue tra un comando e l'altro (Sezione 11).

3 Il ciclo di vita del programma

3.1 Dal `main` al ciclo degli eventi

`src/main.rs` fa quattro cose: chiama `create_global_env::<GapBuffer>()`, attiva il file di log `rsedit.log` nella directory corrente, apre il file passato come primo argomento valutando `(find-file PERCORSO)` e, se lo standard input è un terminale, entra in `tui_main`. Un errore di avvio dell'interprete viene stampato e il programma esce; un errore di `find-file` finisce nell'area dei messaggi.



3.1.1 I passi di `create_global_env`

1. `EditorState::new()` riempie la keymap globale con le poche associazioni scritte in Rust (`fill_default_keymaps`), crea ogni compartimento con il suo `Arc<RwLock<...>>`, avvia lo scheduler di sfondo e gli consegna i due lavori che girano sempre: `Highlighter` e `Prescanner`, ogni 40 ms.
2. `bootstrap_vm` crea l'ambiente radice con le 86 primitive di base e verifica l'interprete.
3. `lisp-path` diventa ("`<directory dell'eseguibile>/data/lisp`").
4. Le impostazioni ricevono il valore iniziale, così che `(setq ...)` in una configurazione modifichi un valore che esiste già: `mouse-mode` (`nil`), `after-resize-hook` (`nil`), `echo-message-timeout` (5), `minibuffer-width` (60), `minibuffer-height` (3), `mode-line-format`, `window-separator` (`|`). Viene creato il modo `fundamental-mode`.
5. `install_primitives`, `install_minibuffer` e `install_isearch` registrano le primitive, i comandi, il modo `minibuffer-mode` e il modo `isearch-mode`.
6. `rsedit-path` riceve il percorso dell'eseguibile.
7. Se manca, `init.lisp` viene creato con `DEFAULT_INIT_LISP` (vedi sotto).
8. Lo scheduler riceve `FileWatcher` e `AutoSaver`, con gli intervalli letti dalle variabili (`watch-file-interval`, `auto-save-interval`).
9. `eval_file` valuta `init.lisp`, che carica i moduli Lisp.

3.1.2 Dove sta la configurazione

Fonte, in ordine di precedenza	Directory
<code>RSEDIT_CONFIG_DIR</code>	usata così com'è, su ogni piattaforma
<code>XDG_CONFIG_HOME</code> (solo Unix)	<code>\$XDG_CONFIG_HOME/rsedit</code>
la piattaforma	<code>%APPDATA%\rsedit</code> su Windows, <code>\$HOME/.config/rsedit</code> altrove (anche su macOS)

Il file `init.lisp` predefinito carica, in quest'ordine: `commands`, `debug`, `common-keymaps`, `indent`, `minibuffer`, poi i moduli facoltativi `rust-mode`, `risp-mode`, `cc-mode`, `dired`, `completion`, `preview`, `clipboard`, `electric-pair`, `buffer-list`, `shell`, `manpage`, `help`, `occur`, `compile`, `theme`, `completion-at-point`, e infine imposta `dired-sort` a `'type` e `mouse-mode` a `t`. L'ordine conta: `commands` definisce `defcommand`, con cui sono scritti gli altri, e `debug` definisce `message` e `report-error`.

Il file viene scritto solo se manca: chi ha già un `init.lisp` deve aggiungere a mano le righe `(eval-file ...)` dei moduli nuovi (→ [AVV-5](#)).

3.1.3 eval_file

`eval_file(nome, env)` cerca prima `nome` come percorso. Se non esiste e il nome non contiene `/`, `\` o `.`, cerca `nome.lisp` in ogni directory di `lisp-path`, in ordine. Il contenuto viene avvolto in `(progn ...)`, letto con *una sola* chiamata a `Parser::next` e valutato dentro uno scope di carburante (`begin_command`). Un file mancante produce una riga di log e il valore `nil`.

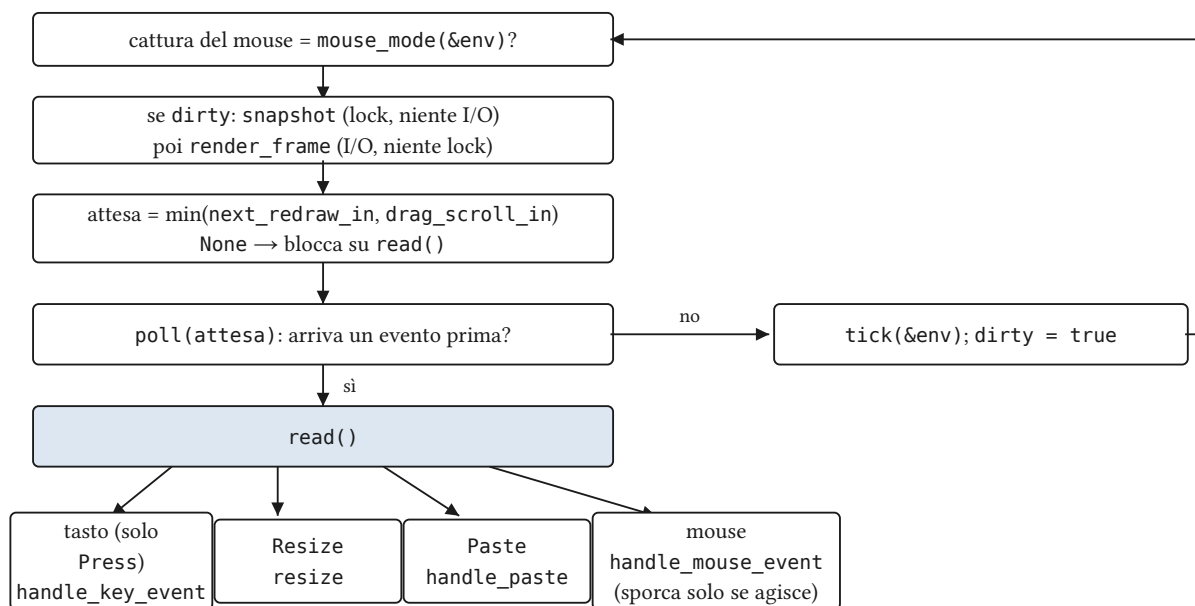
da `editor/boot.rs` — estratto semplificato

```
let content = format!("{}", /* il testo del file, o di nome.lisp in lisp-path */);
let ast = if let Ok(ast) = Parser::new(&content).next() { ast } else {
    return Ok(ELispExp::nil()); // errore di sintassi: nessun messaggio
};
let _command = self.begin_command(); // uno scope di carburante
Ok(ELispExp::proper_list(vec![eval(&ast, env.clone(), self)?]))
```

L'involucro `(progn ...)` fa del file un'unica espressione, e questo ha tre conseguenze. Una parentesi chiusa di troppo chiude la `progn` in anticipo e il resto del file non viene letto; una parentesi mancante, o un commento sull'ultima riga senza a capo finale (la parentesi aggiunta finisce dentro il commento), lascia la `progn` aperta e il lettore fallisce. In entrambi i casi `eval_file` restituisce `nil` senza messaggi (→ [AVV-2](#)). Il primo errore di *valutazione*, invece, interrompe il file da quel punto e risale come errore.

3.2 Il ciclo degli eventi della TUI

`tui_main` (`src/tui.rs`) attiva la modalità `raw` e l'incolla «bracketed», rileva la profondità di colore (`COLORTERM`, `TERM`), comunica le dimensioni iniziali in `frame-width` e `frame-height`, e poi gira finché `is_running()` è vero.



Il cuore del ciclo, senza la gestione della cattura del mouse:

da `../../src/tui.rs` — estratto semplificato

```
while state.is_running() {
    if dirty {
        let (cols, rows) = terminal::size()?;
        frame = state.snapshot(&env, cols as usize, rows as usize); // lock, niente I/O
        render_frame(&frame, depth)?; // I/O, niente lock
        dirty = false;
    }
    let wake = [state.next_redraw_in(&env, &frame), state.drag_scroll_in()]
        .into_iter().flatten().min();
    if let Some(remaining) = wake && !poll(remaining)? {
        state.tick(&env); // l'attesa è scaduta: callback dovute, scorrimento
        dirty = true;
        continue;
    }
    match read()? {
        Event::Key(k) => if let Some(ev) = translate_key(k)
            && k.kind == KeyEventKind::Press {
            state.handle_key_event(ev, &env); dirty = true;
        },
        Event::Resize(w, h) => {
            state.resize(env.clone(), w as usize, h as usize); dirty = true;
        }
        Event::Paste(text) => { state.handle_paste(text, &env); dirty = true; }
        Event::Mouse(m) => if let Some(ev) = translate_mouse(m)
            && state.handle_mouse_event(ev, &env) { dirty = true; },
        _ => (),
    }
}
```

Tre dettagli contano. Il ridisegno avviene solo se qualcosa è cambiato (`dirty`): un terminale che riporta il mouse manda decine di movimenti al secondo, e ridisegnare per ognuno farebbe sfarfallare lo schermo. L'attesa è guidata dall'editor: se nulla è in sospeso `next_redraw_in` risponde `None` e il ciclo si blocca su `read()` senza consumare CPU. E `tick` viene chiamato solo quando l'attesa scade, non a ogni giro.

Un tasto che `translate_key` non sa tradurre — `Delete`, `Home`, `End`, `PageUp`, `PageDown`, `Insert`, i tasti funzione — viene soltanto registrato nel log: `KeyCode` non ha varianti per loro (→ [CMD-2](#)). Shift con un carattere diventa la maiuscola; `BackTab` diventa `Tab` con Shift.

All'uscita normale la TUI disattiva la cattura del mouse e l'incolla, pulisce lo schermo e lascia la modalità raw. Lo stesso ripristino non avviene se il ciclo esce per un errore di I/O o per un panic (→ [TUI-2](#)). La TUI disegna sullo schermo principale del terminale, non su quello alternativo (→ [TUI-8](#)).

4 Lo stato: `EditorState` e i compartimenti

4.1 La facciata

`EditorState<B: BufferTrait>` è una struttura di `Arc`: clonarla costa poco e ogni clone vede lo stesso stato. I suoi campi sono privati, e ognuno è o un *compartimento* di `managers/` sotto un `RwLock`, o un dato piccolo con un lock o un atomico proprio.

Campo	Tipo	Contenuto
<code>buffers</code>	<code>Buffers</code>	Tabella nome → <code>Arc<RwLock<Buffer>></code> , buffer corrente, ordine di recente uso.
<code>windows</code>	<code>Windows</code>	Albero delle finestre affiancate, finestre flottanti, focus, prossimo id, trascinamento, ultimo clic.
<code>modes</code>	<code>Modes</code>	Registro dei modi maggiori, keymap globale, completamenti globali, modi per estensione, keymap transitoria, tasti di ripetizione, hook globali.
<code>commands</code>	<code>Commands</code>	Registro dei comandi, pila dei comandi in attesa di argomenti, argomento prefisso, ultimo comando e sua forma.
<code>kill_yank</code>	<code>KillYank</code>	Kill ring, ultimo yank, flag «il comando precedente ha ucciso/incollato».
<code>macros</code>	<code>Macros</code>	Macro in registrazione, ultima macro, contatore.
<code>history</code>	<code>History</code>	Storia dei prompt per chiave (100 voci per chiave predefinite).
<code>log</code>	<code>Log</code>	Righe diagnostiche e file a cui sono copiate.
<code>runtime</code>	<code>Runtime</code>	Colonna obiettivo, isearch e sostituzione in corso, <code>FuelMeter</code> , pila delle chiamate per i backtrace, tema, generazioni dei worker.
<code>echo_message</code>	<code>RwLock<EchoMessage></code>	Testo dell'area messaggi con l'istante in cui è stato scritto.
<code>pending_keys</code>	<code>RwLock<Vec<KeyEvent>></code>	Tasti di una sequenza non ancora completa.
<code>owed</code>	<code>RwLock<Vec<OwedCallBack>></code>	Callback chieste dai lavori di sfondo e non ancora eseguite.
<code>current_results</code>	<code>RwLock<Option<String>></code>	Il buffer di risultati che <code>next-error</code> percorre.
<code>worker_mailbox</code>	<code>Sender<WorkerMessage></code>	Canale verso lo scheduler.
<code>running</code>	<code>AtomicBool</code>	Falso dopo <code>quit</code> .
<code>shell_commands</code> , <code>background_work</code>	<code>AtomicUsize</code>	Quanti comandi di shell e lavori di sfondo stanno cambiando lo schermo.
<code>command_errors</code>	<code>AtomicUsize</code>	Quanti comandi sono falliti: la riproduzione di una macro si ferma al primo.

`EditorState` implementa `Debug` e `PartialEq` solo perché `LispContext` li richiede; nessuno dei due è pensato per essere usato (→ [DOC-2](#)).

4.2 Le regole dei compartimenti

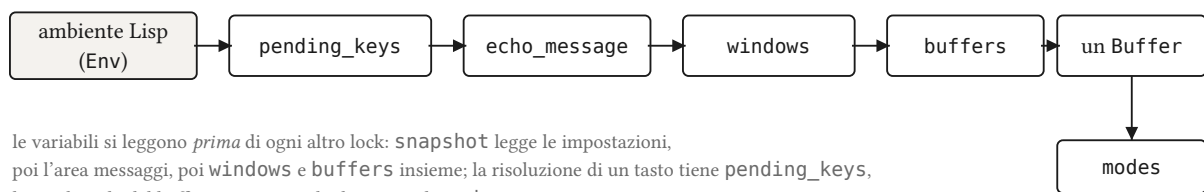
Il commento in testa a `managers/mod.rs` le enuncia; il codice le rispetta.

Regola	Perché
Un compartimento non raggiunge un altro compartimento.	Riceve i valori che gli servono e restituisce risposte (<code>Windows::scroll_focused</code> riceve un numero di righe, non un buffer). Così solo la facciata tiene due lock insieme.
Non consegna guardie.	Si accede allo stato solo con una closure (<code>windows(w ...)</code>), e il lock non sopravvive alla domanda.
Non valuta mai.	Nessun compartimento riceve un <code>Env</code> . Può conservare valori Lisp (il corpo di un hook), mai eseguirli: Lisp può rientrare nell'editor da qualunque primitiva.
Risponde con un verdetto, non con un effetto.	Quando un'operazione ha conseguenze altrove (un buffer da ripuntare, una finestra da rifocalizzare) restituisce un enum come <code>BufferRemoved</code> o <code>WindowRemoved</code> , e la facciata agisce.
Non fa nulla di lento con il lock preso.	Niente disco, rete o attese. <code>Log::record</code> restituisce il file e la scrittura avviene dopo aver rilasciato il lock.

La regola che nasce da tutte queste, e che vale anche fuori dai compartimenti, è una sola: *mai tenere un lock attraverso una chiamata all'interprete*. `run_hook` copia la lista degli hook e rilascia `modes` prima di valutarli; `command_specs` restituisce dati posseduti perché `call-interactively` valuta Lisp subito dopo. Un `RwLock` non è rientrante: un hook che chiama `add-hook` mentre lo stesso thread tiene il lock di lettura del registro si blocca per sempre, ogni volta.

4.3 L'ordine dei lock

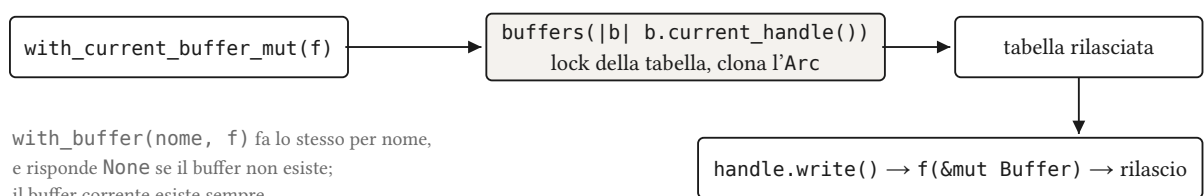
Quando un'operazione deve tenere più lock insieme, li prende in quest'ordine.



le variabili si leggono *prima* di ogni altro lock: `snapshot` legge le impostazioni, poi l'area messaggi, poi `windows` e `buffers` insieme; la risoluzione di un tasto tiene `pending_keys`, legge il modo del buffer corrente e solo dopo prende `modes`.

`buffers` ha una particolarità: nel percorso comune non viene tenuto mentre si blocca un buffer. `with_buffer` e `with_current_buffer` clonano l'`Arc` del buffer dalla tabella, *rilasciano* la tabella e solo allora bloccano il buffer. Così chi scrive in un buffer non chiude fuori dalla tabella gli altri thread — l'evidenziatore e lo scanner la percorrono mentre l'utente scrive. Soltanto `snapshot` tiene la tabella e i singoli buffer insieme, nell'ordine indicato.

4.4 Accesso ai buffer



`with_buffer(nome, f)` fa lo stesso per `nome`, e risponde `None` se il buffer non esiste; il buffer corrente esiste sempre.

`Buffers` non è mai vuoto: nasce con `*scratch*` corrente. Le funzioni che cambiano quale buffer è corrente passano tutte da `make_current`, che rifiuta un nome assente dalla tabella; questa invariante esiste perché una finestra rimasta a mostrare un buffer cancellato aveva già fatto crollare l'editor.

4.5 L'editor come contesto dell'interprete

`EditorState<B>` è il `T: LispContext` dell'interprete:

Metodo del trait	Implementazione
<code>consume_fuel(n)</code>	<code>FuelMeter</code> di <code>Runtime</code> (clonato fuori dal lock); <code>Exhausted</code> diventa <code>OutOfFuel</code> .
<code>log_diagnostic(msg)</code>	<code>Log::record</code> sotto lock, poi la scrittura sul file senza lock.
<code>begin_unwind()</code>	Alza il carburante ad almeno 100.000 passi ($\rightarrow$ <code>INT-1</code>).
<code>begin_thread_evaluation()</code>	<code>arm_thread()</code> : il nuovo thread riceve il budget configurato.
<code>push_call_frame /</code> <code>pop_call_frame /</code> <code>call_frame_depth /</code> <code>truncate_call_frames</code>	La pila <code>call_stack: Vec<String></code> di <code>Runtime</code> , che <code>report_error</code> legge come backtrace.

Lo scope di carburante dell'editor è `begin_command()`: lo aprono `run_command_form`, ogni hook, `report_error`, ogni callback dovuta, ogni funzione di `after-resize-hook` ed `eval_file`. Solo l'apertura più esterna ricarica il budget (10.000.000 di passi), quindi un `eval-file` dentro un comando consuma il budget del comando. Gli hook, invece, sono valutati dopo che lo scope del comando si è chiuso: ognuno riparte con il budget pieno.

5 Dal tasto al comando

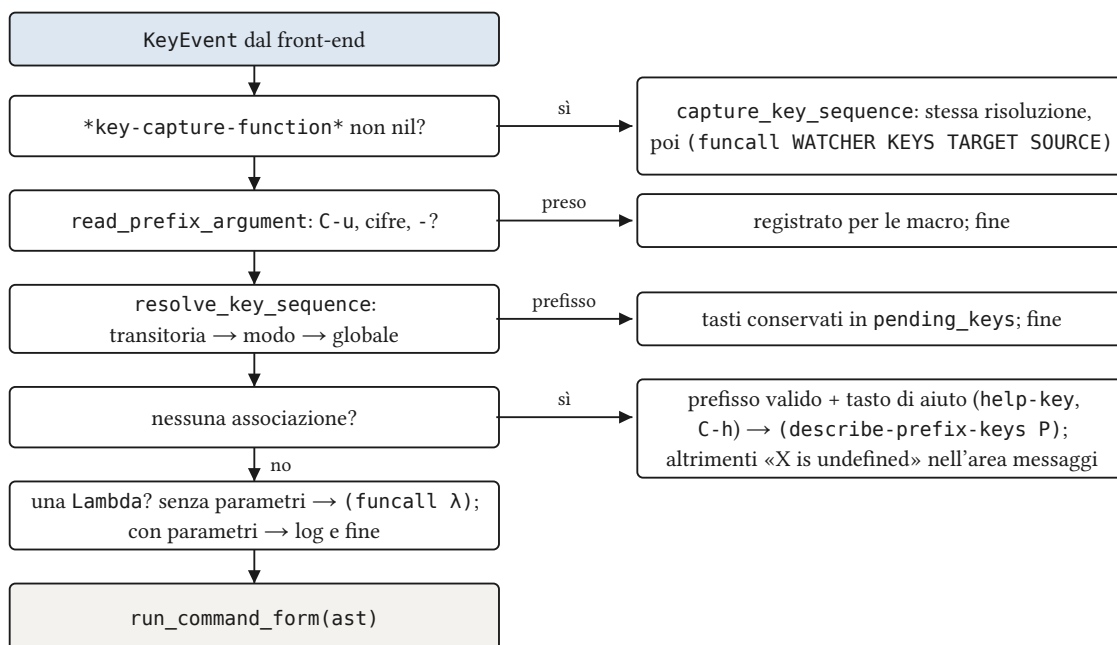
5.1 Il vocabolario dei tasti

Un `KeyEvent` è un `KeyCode` (`Char(c)`, `Backspace`, `Enter`, `Esc`, `Tab`, le quattro frecce) più i modificatori `ctrl`, `alt`, `shift`. Nelle associazioni i tasti si scrivono come in Emacs: `C-x`, `M-x`, `C-M-f`, `S-<tab>`, con i nomi `<ret>`, `<esc>`, `tab`, `<backtab>`, `<backspace>`, `<space>`, `<left>`, `<right>`, `<up>`, `<down>`. Una sequenza è una stringa di tasti separati da spazi: `"C-x C-f"`. `parse_key_sequence` (in `primitives/`) legge queste stringhe; `describe_keys` fa il contrario.

La keymap globale scritta in Rust contiene pochissimo: `M-x` (`command-execute-prompt`), le frecce, Invio e i caratteri stampabili da `' '` a `'~'`, legati a `(self-insert "c")`. Tutto il resto — `C-x C-f`, `C-s`, `C-k`, ... — viene da `common-keymaps.lisp` e dagli altri moduli.

Solo i caratteri ASCII stampabili sono legati a `self-insert`, e non esiste un'associazione di ripiego per gli altri: un carattere come «è» arriva alla risoluzione senza un comando e produce `è is undefined` (→ `CMD-1`).

5.2 `handle_key_event`



I tasti vengono registrati per la macro di tastiera in corso anche quando non eseguono nulla: una macro deve riprodurre ciò che è stato premuto, compresi `C-u 5` e i tasti non associati.

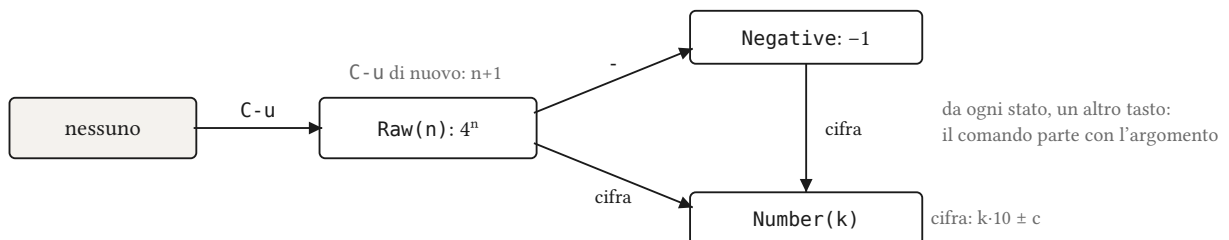
Il front-end chiama `handle_key_event` solo per gli eventi di pressione; l'incolla arriva da `handle_paste` come un unico comando (`insert-pasted-text TEST0`), che passa anch'esso da `run_command_form`.

In codice, la funzione è una sequenza di uscite anticipate:

```
pub fn handle_key_event(&self, event: KeyEvent, env: &Arc<Env<EditorState<B>>>) {
    if let Some(watcher) = env.get_variable(KEY_CAPTURE_FUNCTION) && watcher.is_truthy() {
        self.record_keys(&[event.clone()], None);
        self.capture_key_sequence(event, watcher, env); // read-key-sequence
        return;
    }
    if self.read_prefix_argument(&event) { // C-u, cifre, -
        self.record_keys(&[event], None);
        return;
    }
    let mut sequence = self.pending_key_events();
    sequence.push(event.clone());
    let mut ast = match self.resolve_key_sequence(event.clone()) {
        Dispatch::Run(ast) => { self.record_keys(&sequence, command_named_by(&ast)); ast }
        Dispatch::Nothing => return, // un prefisso: si aspetta
        Dispatch::Unbound { described, prefix, last } => {
            /* prefisso + tasto di aiuto → describe-prefix-keys; altrimenti
               "X is undefined" nell'area messaggi */
            return;
        }
    };
    if let ELispExp::Lambda(ref lambda) = ast { // un tasto legato a una lambda
        if !lambda.params.is_empty() { /* log */ return; }
        ast = ELispExp::form(vec![ELispExp::symbol("funcall"), ast]);
    }
    self.run_command_form(ast, env);
}
```

5.3 L'argomento prefisso

C-u e le cifre non arrivano mai alle keymap: `Commands::read_prefix_argument` li prende per primo. Lo stato è la coppia `prefix_arg: Option<PrefixArg>` e `reading_prefix: bool`.



Un tasto qualunque che non fa parte dell'argomento chiude la lettura (`reading_prefix = false`) ma lascia l'argomento al comando che segue. Il comando lo legge tramite `call-interactively` (le specifiche `p` e `P`); dopo ogni comando, riuscito o fallito, `clear_prefix_argument` lo cancella. Non esistono le forme M-4 o C-4 di Emacs. Mentre l'argomento e la sequenza sono a metà, `pending_input()` li descrive (C-u 4 C-x-) e la TUI li mostra a destra nella riga dei messaggi.

I caratteri sono legati a forme che portano già il loro argomento, (`self-insert "x"`); una forma così non passa da `call-interactively`, e quindi non riceve l'argomento prefisso: C-u 3 x inserisce una sola «x» (→ CMD-5).

5.4 La risoluzione: `resolve_key_sequence`

Il tasto si aggiunge a `pending_keys`; poi, con un solo lock di lettura su `modes`, la sequenza viene cercata in tre keymap, in ordine:

1. la *keymap transitoria*, se installata;

2. la keymap del modo maggiore del buffer corrente;
3. la keymap globale.

da `editor/keys.rs` — estratto semplificato

```
let mut pending = self.pending_keys.write().unwrap();
pending.push(event);
let current_mode = self.with_current_buffer(|buf| buf.current_mode.clone());
let bound = self.modes(|modes| {
    if let Some(map) = modes.transient() {
        match (
            map.keymap.get(&pending).cloned(),
            map.keymap.is_prefix(&pending),
            map.on_unbound,
        ) {
            (Some(ast), _, _) => return Bound::Command(ast),
            (None, true, _)   => return Bound::Prefix,
            (None, false, OnUnbound::Refuse) => return Bound::Refused,
            (None, false, OnUnbound::Pass)   => {},
            (None, false, OnUnbound::Release) => release_transient = true,
        }
    }
    // prima il modo, poi la keymap globale
    let hit = modes.binding_below_transient(Some(&current_mode), &pending);
    let prefix = modes.is_prefix_below_transient(Some(&current_mode), &pending);
    match (hit, prefix) {
        (Some(binding), _) => Bound::Command(binding.target),
        (None, true)       => Bound::Prefix,
        (None, false)      => Bound::Unbound,
    }
});
if !matches!(bound, Bound::Prefix) { pending.clear(); }
```

Una sequenza è cercata *intera*: `pending_keys` contiene già i tasti precedenti, e la tabella di una keymap associa sequenze di tasti, non tasti singoli. Un risultato `Prefix` lascia la sequenza in `pending_keys`, in attesa del tasto successivo; ogni altro risultato la svuota. Un modo che lega un prefisso tiene viva la sequenza anche se solo la globale la completa. Le stesse due funzioni (`binding_below_transient`, `is_prefix_below_transient`) rispondono anche a `key-binding` e ai comandi di aiuto, così ciò che l'aiuto riporta e ciò che l'editor esegue non possono divergere.

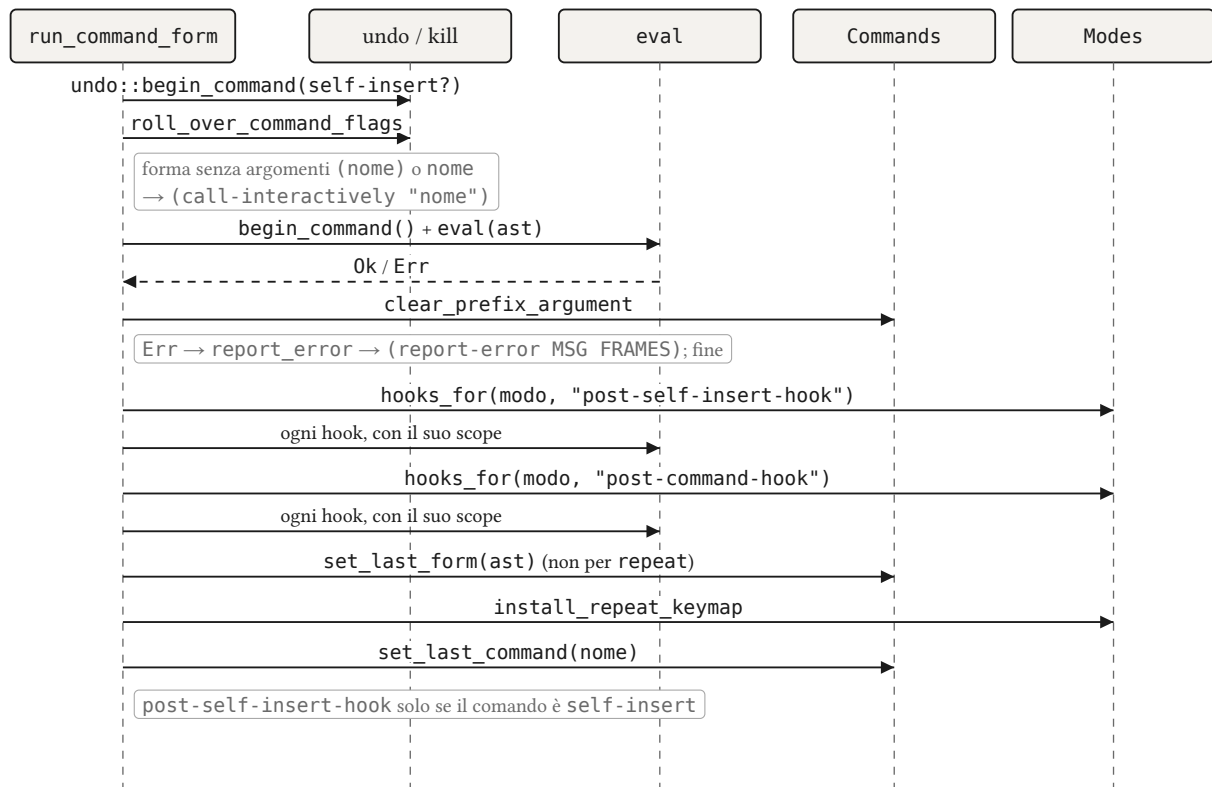
La keymap transitoria (`TransientKeymap`) è installata da una primitiva o dal meccanismo di ripetizione e dice che cosa fare di un tasto che non conosce:

OnUnbound	Comportamento	Uso tipico
<code>Refuse</code>	Il tasto è rifiutato in silenzio; la mappa resta. Il messaggio della mappa occupa la riga dei messaggi.	Una domanda (<code>save-some-buffers</code> , <code>query-replace</code>): si risponde solo con i suoi tasti.
<code>Pass</code>	Il tasto prosegue verso le altre keymap; la mappa resta.	La striscia dei completamenti: si può continuare a scrivere.
<code>Release</code>	Il tasto prosegue; la mappa viene tolta.	La ripetizione (<code>C-x o o o</code>): un'offerta che si può ignorare.

Dopo ogni comando, `install_repeat_keymap` controlla se il comando ha un *tasto di ripetizione* dichiarato (`define-repeat-key`) e in tal caso installa una mappa `Release` che lo associa al comando stesso.

5.5 `run_command_form`: l'esecuzione di un comando

Tutto ciò che il tasto, l'incolla, la cattura e la riproduzione di macro fanno eseguire passa da qui.



da `editor/commands.rs` – estratto semplificato

```

pub(super) fn run_command_form(&self, mut ast: ELispExp<B>, env: &Arc<Env<EditorState<B>>>) {
    let bound_command = /* Some(nome) se ast è `nome` o `(nome)`, cioè senza argomenti */;
    let this_command = /* il simbolo in testa, se c'è */;
    undo::begin_command(this_command == Some("self-insert")); // nuova epoca
    self.roll_over_command_flags(); // kill ring
    if let Some(name) = bound_command {
        ast = ELispExp::form(vec![
            ELispExp::symbol("call-interactively"), ELispExp::string(name),
        ]);
    }
    let outcome = { let _command = self.begin_command(); eval(&ast, env.clone(), self) };
    self.clear_prefix_argument();
    if let Err(e) = outcome {
        self.report_error(&format!("{:?} {:?}", ast, e), env); return;
    }
    let mode = self.with_current_buffer(|buf| buf.current_mode.clone());
    if this_command == Some("self-insert") {
        self.run_hook(&mode, "post-self-insert-hook", env);
    }
    self.run_hook(&mode, "post-command-hook", env);
    if this_command != Some("repeat") { self.commands_mut(|c| c.set_last_form(Some(ast))); }
    self.install_repeat_keymap(this_command);
    self.set_last_command(this_command);
}

```

1. *Undo*. `undo::begin_command` assegna al thread una nuova epoca; tutte le modifiche del comando finiranno in un solo gruppo (Sezione 8.4).
2. *Flag del kill ring*. «Il comando precedente ha ucciso» diventa vero o falso *prima* che il comando parta, così una serie di `C-k` si accumula in una sola voce del kill ring e un comando che fallisce non lascia il flag acceso.

3. *Instradamento*. Una forma senza argomenti diventa una chiamata a `call-interactively` con il nome del comando, così l'editor raccoglie gli argomenti dichiarati. Una forma che porta già i suoi argomenti — `(self-insert "a")` — resta com'è: il percorso della digitazione non paga nulla.
4. *Esecuzione*. `eval` dentro uno scope di carburante.
5. *Argomento prefisso* cancellato, anche se il comando è fallito.
6. *Errore*. `report-error` incrementa `command_errors`, prende la pila delle chiamate come backtrace, la svuota e chiama la funzione Lisp `report-error` (definita in `debug.lisp`: messaggio nell'area messaggi e, se `debug-on-error`, una finestra `*Backtrace*`). Senza quella funzione scrive nel log. Dopo un errore non girano gli hook e `last-command` non cambia.
7. *Hook*. Prima `post-self-insert-hook` (solo per `self-insert`: qui si aggancia `electric-pair`), poi `post-command-hook`. Ogni lista è la concatenazione degli hook del modo e di quelli globali, copiata con un solo lock; un hook che fallisce viene registrato nel log e non ferma gli altri.
8. *Memoria*. La forma eseguita diventa quella di `repeat`; si offre il tasto di ripetizione; `last-command` riceve il nome.

5.6 La cattura di una sequenza

`read-key-sequence` arma `*key-capture-function*`. Il tasto successivo segue la stessa risoluzione di un tasto normale — la sequenza a metà si vede come sempre — ma al posto di eseguire il comando viene chiamata la funzione con la sequenza, la forma che sarebbe stata eseguita (citata) e il nome della keymap che la fornisce, o `"prefix-argument"` per `C-u`. È così che `describe-key` e `zap-to-char` leggono un tasto: non esiste in rsedit un «leggi un tasto» che blocchi, perché i tasti arrivano uno per giro del ciclo degli eventi.

6 Comandi e argomenti

6.1 Che cos'è un comando

Per l'interprete un comando non esiste: è una funzione qualunque. Diventa un comando quando ha una voce nel registro di `Commands` (`register_command`), che associa al nome l'elenco degli argomenti da chiedere. Una primitiva diventa comando registrandosi con `into.command(nome, puntatore, specifiche, DOC)` invece che con `into.function`; una funzione Lisp con la macro `defcommand` di `commands.lisp`:

```
(defcommand greet (who) ("sGreet whom: ")
  "Say hello to WHO."
  (message "Hello, %s" who))
```

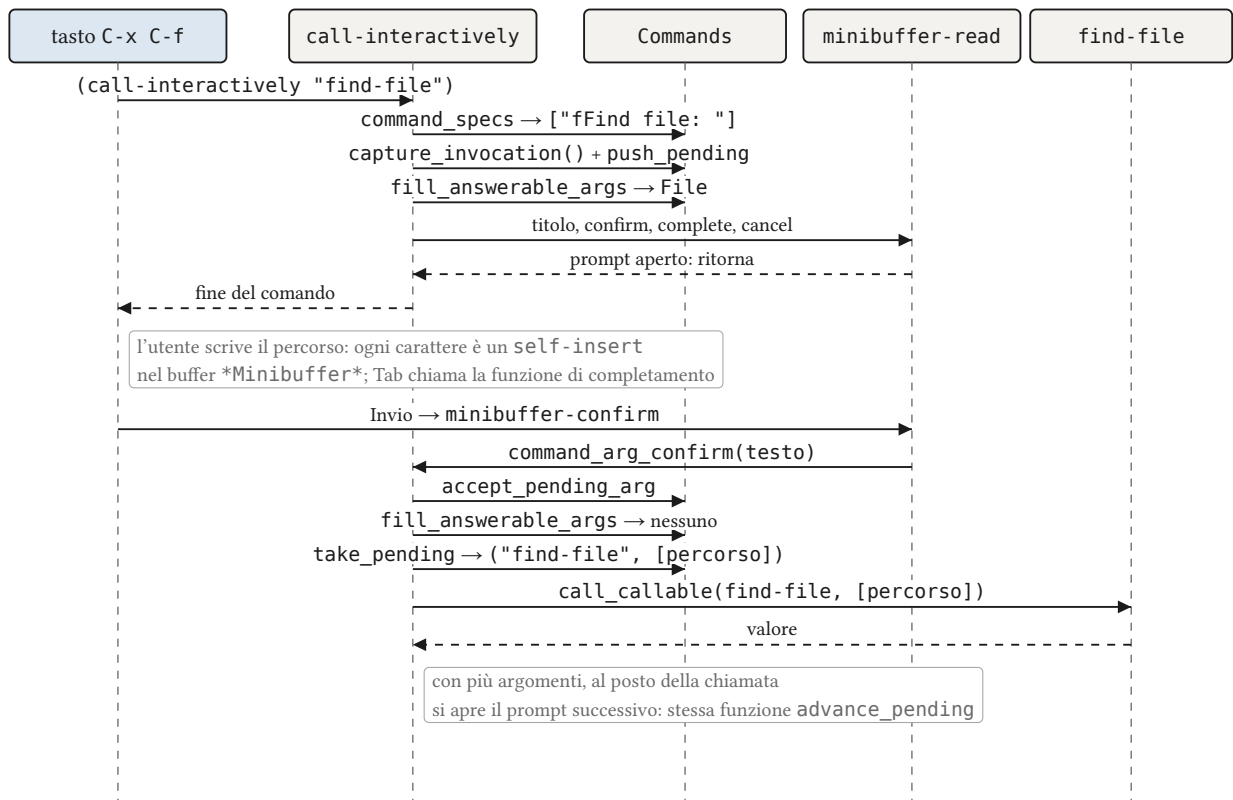
`M-x` completa solo sui nomi registrati; una funzione non registrata si può comunque legare a un tasto, e viene chiamata senza argomenti. Le specifiche sono analizzate una sola volta, alla registrazione: un codice sbagliato è un errore all'avvio, non la prima volta che il comando gira.

6.2 Le specifiche degli argomenti

Codice	ArgSpec	Valore passato al comando
s	String	Il testo digitato nel prompt.
n	Number	Il testo convertito in numero; un testo non numerico diventa 0 (→ CMD-6).
b	Buffer	Un nome di buffer, con completamento sui buffer vivi (escluso <code>*Minibuffer*</code>).
f	File	Un percorso, completato un componente alla volta.
p	Count	L'argomento prefisso come numero, 1 se assente. Non chiede nulla.
P	RawCount	L'argomento grezzo: <code>nil</code> , <code>(4)</code> per <code>C-u</code> , un numero, o il simbolo <code>-</code> . Non chiede nulla.
r	Region	<i>Due</i> valori, inizio e fine della regione attiva. Senza regione il comando è rifiutato prima di aprire qualunque prompt.

6.3 `call-interactively` e la catena degli argomenti

Il minibuffer non blocca: apre un prompt e ritorna, e la risposta arriva con un tasto successivo. Raccogliere due argomenti non può quindi essere un ciclo: è una catena, e lo stato tra un anello e l'altro sta nella pila dei `PendingCommand` di `Commands`.



Invocation — argomento prefisso e regione — è catturata quando il comando inizia, non quando l'ultimo argomento arriva. È ciò che l'utente intendeva premendo `C-u 4 M-x foo`, e nel frattempo il prompt ha cambiato buffer e l'argomento prefisso è già stato cancellato.

Il titolo del prompt dice chi sta chiedendo: `find-file - Find file:` o, con più argomenti, `query-replace (2/2) - With:`. Il prompt viene aperto chiamando `minibuffer-read` per nome, così un minibuffer alternativo serve anche gli argomenti dei comandi. Annullare il prompt (`command_arg_cancel`) toglie il comando dalla pila: un elenco di argomenti incompleto non arriva mai a un comando. Un nuovo `call-interactively` senza minibuffer aperto svuota la pila dagli orfani rimasti da prompt chiusi in altro modo.

Il completamento degli argomenti `b` e `f` è fatto in Rust; una funzione in `*command-arg-completion-function*`, se impostata, lo sostituisce per tutti i tipi ricevendo il tipo come simbolo (`buffer`, `file`, ...) e il prefisso.

7 Minibuffer e ricerca incrementale

7.1 Il minibuffer

Il meccanismo del minibuffer è in Rust (`feature/minibuffer.rs`, `primitives/minibuffer.rs`) perché troppe cose ne dipendono — `M-x`, `M-:`, gli argomenti dei comandi — per affidarlo a un file Lisp che potrebbe mancare. L'aspetto, invece, è sostituibile: `minibuffer-read` chiama la funzione contenuta in `*minibuffer-read-function*`, che per impostazione predefinita è `default-minibuffer-prompt`.

```
(minibuffer-read PROMPT ON-CONFIRM ON-CHANGE ON-CANCEL [MODE [HISTORY-KEY]])
```

Argomento	Significato
ON-CONFIRM	Chiamata con il testo finale quando si preme Invio, <i>dopo</i> la chiusura del prompt.
ON-CHANGE	Nonostante il nome, è la funzione di completamento: riceve il testo e restituisce i candidati. È chiamata al primo Tab dopo ogni modifica; i Tab successivi scorrono i candidati, o li consegnano a <code>*completion-read-function*</code> se impostata (è ciò che fa <code>completion.lisp</code>).
ON-CANCEL	Chiamata senza argomenti quando si preme Esc.
MODE	Il modo del buffer del prompt (predefinito <code>minibuffer-mode</code>); <code>isearch-mode</code> per la ricerca.
HISTORY-KEY	La chiave della storia (<code>M-p</code> , <code>M-n</code>); per default il testo del prompt, così ogni prompt ha una storia senza chiederla. Fino a <code>history-length</code> voci, 100 per default.

`default-minibuffer-prompt` salva le callback in variabili globali (`*minibuffer-on-confirm*`, `*minibuffer-on-change*`, `*minibuffer-on-cancel*`, `*minibuffer-previous-buffer*`, ...) e apre una finestra flottante con bordo sul buffer `*Minibuffer*`, *al centro* del frame, larga `minibuffer-width` (60) e alta `minibuffer-height` (3) e ridotta se il terminale è più piccolo. Il centro è scelto perché il fondo dello schermo è occupato da altre strisce (i completamenti, la compilazione); alcuni commenti descrivono ancora un prompt agganciato in basso (→ [DOC-1](#)). Ogni prompt usa lo stesso buffer, `*Minibuffer*`: due prompt aperti insieme lo condividono (→ [MIN-1](#)). `C-g` nel prompt resta `keyboard-quit`, che non lo chiude; lo chiude Esc (→ [MIN-2](#)).

Un worker di sfondo non può aprire un prompt: `minibuffer-read` controlla `in_worker()` e rifiuta con un errore, perché il tasto che risponderebbe arriverebbe in mezzo a ciò che l'utente sta facendo.

7.2 La ricerca incrementale

`C-s` apre un prompt in `isearch-mode` e ricorda dove era il punto. Da lì il lavoro lo fa il `post-command-hook` di quel modo: ogni comando eseguito mentre il prompt è aperto — ogni carattere scritto, ogni cancellazione — provoca un giro di `isearch-update`, che ripete la ricerca e sposta il punto. Non serve un ciclo «leggi un tasto», che in `rsedit` non esiste.

Mentre il prompt è aperto il buffer corrente è il minibuffer. Tutto ciò che riguarda la ricerca lavora quindi sul buffer cercato *per nome*, preso dallo stato della sessione (`Isearch`) in Runtime. Una modifica che usasse il buffer corrente cercherebbe nella riga del prompt; il test `an_isearch_searches_the_file_not_the_prompt` esiste per questo. Lo stato della sessione e l'algoritmo di ricerca sono descritti nel Sezione 9.

8 I buffer

8.1 La struttura `Buffer`

Campo	Contenuto
<code>name</code> , <code>file_path</code>	Nome e file visitato.
<code>text: B</code>	Il testo, tramite <code>BufferTrait</code> (oggi <code>GapBuffer</code>). Contiene anche il punto (il cursore).
<code>is_modified</code> , <code>read_only</code>	Modificato rispetto al disco; rifiuta le modifiche (controllato alle due porte).
<code>current_mode</code>	Il nome del modo maggiore; decide keymap, hook, grammatica e tabella sintattica.
<code>undo</code>	<code>UndoHistory</code> : gruppi fatti e disfatti (Sezione 8.4).
<code>mark:</code> <code>Option<Mark></code>	Posizione e flag <code>active</code> ; la regione è tra mark attivo e punto.
<code>overlays</code>	<code>OverlayTable</code> : facce su intervalli, per categoria, che seguono le modifiche.
<code>virtual_text</code>	Testo mostrato in una posizione senza essere nel buffer (suggerimenti, anteprime).
<code>version</code>	Incrementata da ogni modifica: un risultato calcolato su un'altra versione viene rifiutato.
<code>syntax</code> , <code>scan</code>	Cache della colorazione e della scansione delle espressioni.
<code>file_stamp</code> , <code>stale</code>	Timbro (istante di modifica e lunghezza) del file all'ultima lettura o scrittura; flag «cambiato sotto di noi».
<code>auto_saved_at</code>	Versione scritta nell'ultimo salvataggio automatico.
<code>data</code>	<code>HashMap<String, Box<dyn Any>></code> : dati che altre parti appendono al buffer e che muoiono con lui.
<code>local_keymap</code>	Dichiarato ma mai usato (→ DOC-2).

8.2 `BufferTrait` e `GapBuffer`

`BufferTrait` è l'unica cosa che il resto dell'editor può assumere sul testo: lunghezza, carattere in una posizione, punto in coordinate lineari e in (riga, colonna), conversioni tra le due, spostamenti del punto, inserimento e cancellazione di un carattere *al punto*, righe, `clear`. Gli spostamenti e le conversioni lavorano in caratteri, non in byte.

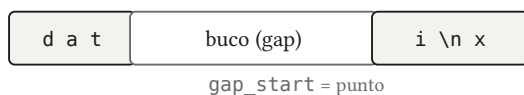
`GapBuffer` tiene un `Vec<char>` con un buco al punto: inserire e cancellare lì costa $O(1)$, spostare il punto costa la distanza percorsa. Accanto al testo tiene un indice delle posizioni fisiche dei `\n`, diviso in prima e dopo il buco: poiché le posizioni fisiche non cambiano quando il buco cresce o si restringe, scrivere un carattere aggiorna l'indice in $O(1)$.

da `buffer/gap_buffer.rs` – estratto semplificato

```
pub struct GapBuffer {
    data: Vec<char>,           // testo e buco nello stesso vettore
    gap_start: usize,          // primo indice del buco = posizione del punto
    gap_end: usize,            // primo indice dopo il buco
    newlines_before: Vec<usize>, // posizioni fisiche dei \n prima del buco
    newlines_after: VecDeque<usize>, // posizioni fisiche dei \n dopo il buco
}

fn insert(&mut self, c: char) {
    if self.gap_start == self.gap_end { self.gap_grow(); } // buco esaurito: raddoppia
    self.data[self.gap_start] = c;
    if c == '\n' { self.newlines_before.push(self.gap_start); }
    self.gap_start += 1;
}
```

Spostare il punto vuol dire spostare il buco: `move_gap` copia con `copy_within` i caratteri che il buco deve scavalcare dall'altra parte, e sposta tra le due metà dell'indice i ritorni a capo che attraversano il buco, correggendone la posizione fisica della larghezza del buco. Un ritorno a capo non cambia mai posizione fisica per un inserimento o una cancellazione al punto, ed è questo che rende $O(1)$ l'aggiornamento dell'indice durante la digitazione. La posizione *logica* del k -esimo ritorno a capo è `newlines_before[k]` se k cade nella prima metà, altrimenti la posizione fisica nella seconda metà meno la larghezza del buco; `line_start(riga)` è la posizione del ritorno a capo precedente più uno. La conversione da posizione lineare a (riga, colonna) del punto, `cursor_pos`, conta invece i caratteri dall'inizio del buffer (→ BUF-2).

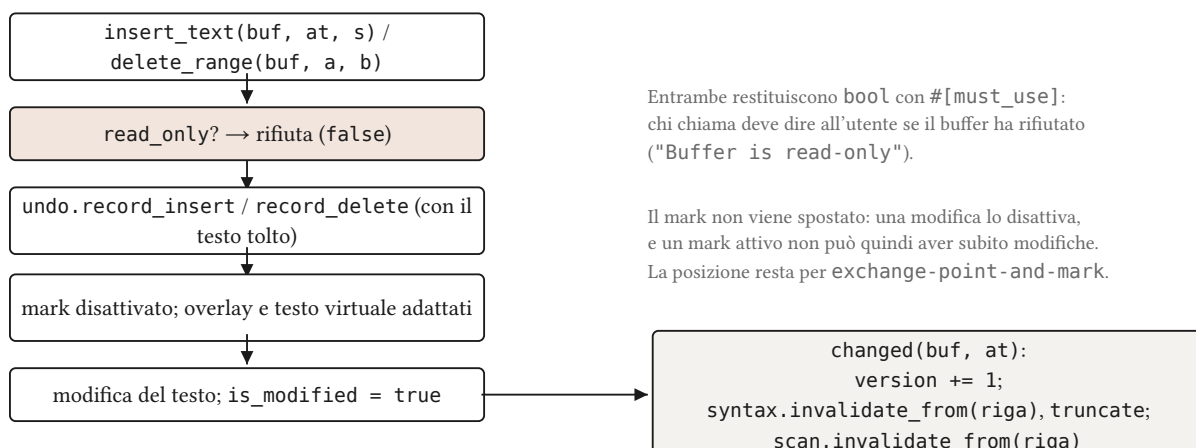


`newlines_before: Vec<usize>`
`newlines_after: VecDeque<usize>` (posizioni fisiche)

un carattere costa 4 byte (`char`); il buco cresce raddoppiando quando si esaurisce

8.3 Le due porte

Ogni primitiva che cambia il testo passa da `edits::insert_text` o da `edits::delete_range`. È la proprietà su cui si reggono undo, overlay, cache e sola lettura: una nuova primitiva di modifica li ottiene tutti per il solo fatto di non poter evitare le porte.



da `primitives/edits.rs` — estratto semplificato

```
pub(crate) fn insert_text<B: BufferTrait>(
    buf: &mut Buffer<B>, at: usize, content: &str,
) -> bool {
    if buf.read_only { return false; }
    if content.is_empty() { return true; }
    let at = at.min(buf.text.len());
    let point = buf.text.cursor_pos_1d();
    let inserted = content.chars().count();
    buf.undo.record_insert(at, inserted, point);           // 1. la storia
    buf.mark = deactivated(buf.mark);                     // 2. la regione
    buf.overlays.adjust_for_insert(at, inserted);         // 3. ciò che è ancorato al testo
    buf.virtual_text.adjust_for_insert(at, inserted);
    undo::apply_insert(&mut buf.text, at, content);      // 4. il testo
    buf.is_modified = true;
    changed(buf, at);                                     // 5. versione e cache
    true
}

fn changed<B: BufferTrait>(buf: &mut Buffer<B>, at: usize) {
    buf.version = buf.version.wrapping_add(1);
    let line = buf.text.cursor_1d_to_2d(at.min(buf.text.len())).0;
    buf.syntax.invalidate_from(buf.version, line);
    buf.syntax.truncate(buf.text.line_count());
    buf.scan.invalidate_from(buf.version, line);
}
```

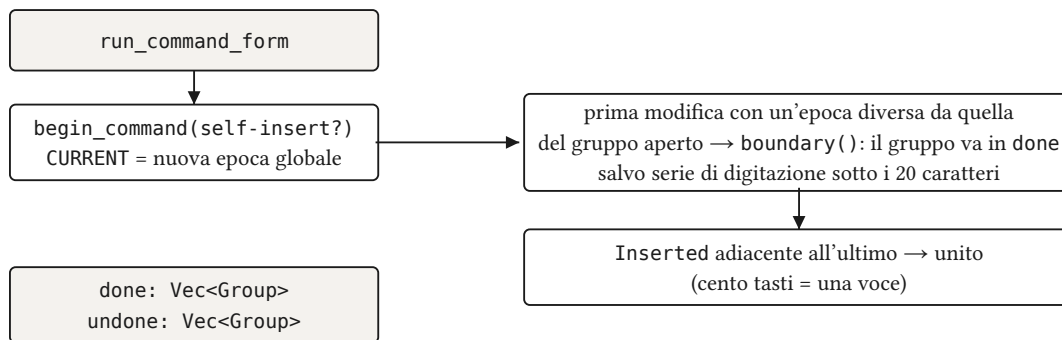
`delete_range` ha la stessa forma. Ordina e limita gli estremi, copia il testo che sta per togliere (tutto il buffer con `to_string`, una porzione con `slice`), lo consegna a `record_delete` — è l'unico modo in cui l'undo potrà rimetterlo — e poi adatta overlay e testo virtuale, cancella e chiama `changed` dalla posizione di partenza. L'ordine conta: la storia registra lo stato *prima* della modifica, gli overlay vengono spostati con gli stessi numeri che descrivono la modifica, e la versione cambia per ultima, così nessun lettore può vedere la versione nuova insieme al testo vecchio (tutto avviene sotto lo stesso lock di scrittura del buffer).

Tre percorsi oggi scavalcano le porte:

- `adopt_text` (lettura di un file, `revert-buffer`, ricarica dal watcher) sostituisce tutto il testo e fa a mano ciò che le porte farebbero: incrementa la versione, invalida entrambe le cache dalla riga 0, svuota overlay, testo virtuale, mark e storia dell'undo (conservandone il limite), riporta il punto alla stessa riga e colonna;
- `set_minibuffer_content`, usata dallo scorrimento dei completamenti, scrive direttamente nel testo del minibuffer (→ MIN-3)
- `undo` e `redo`, che applicano l'inverso di un gruppo direttamente sul testo con `UndoHistory::undo(text)`: non controllano `read_only`, non incrementano la versione, non invalidano le cache e non adattano gli overlay (→ BUF-1).

8.4 Undo

La storia registra *modifiche*, non fotografie: un inserimento è `Inserted { at, len }`, una cancellazione `Deleted { at, text }`. Un gruppo è l'insieme delle modifiche di un comando, con la posizione del punto prima della prima modifica. Disfare applica l'inverso del gruppo e produce il gruppo che lo rifarebbe; rifare è la stessa operazione sull'altra pila.



una nuova modifica svuota undone: nessun albero dei rami

Le due funzioni che decidono i gruppi sono queste:

da `buffer/undo.rs` — estratto semplificato

```

pub(crate) fn begin_command(amalgamating_kind: bool) { // chiamata da run_command_form
    if PINNED.get() > 0 { return; } // dentro una macro: un gruppo solo
    CURRENT.with(|current| {
        let previous = current.get();
        current.set(Command {
            epoch: NEXT_EPOCH.fetch_add(1, Ordering::Relaxed),
            amalgamating_kind,
            joins_previous: amalgamating_kind && previous.amalgamating_kind,
        });
    });
}

fn push(&mut self, change: Change, point: usize) { // ogni modifica passa da qui
    self.undone.clear(); // niente albero dei rami
    let command = current_command();
    if self.open.is_some() && command.epoch != self.open_epoch
        && !(command.joins_previous && self.open_insert_len() < AMALGAMATION_LIMIT)
    {
        self.boundary(); // il gruppo aperto va in `done`
    }
    self.open_epoch = command.epoch;
    self.bytes += change.weight(); // byte di testo cancellato
    let group = self.open.get_or_insert_with(|| Group { changes: Vec::new(), point });
    if let Change::Inserted { at, len } = change
        && let Some(Change::Inserted { at: prev_at, len: prev_len }) =
            group.changes.last_mut()
        && *prev_at + *prev_len == at
    {
        *prev_len += len; // inserimento contiguo: si allunga
        return;
    }
    group.changes.push(change);
}
  
```

Disfare un gruppo vuol dire applicare le sue modifiche al contrario, dall'ultima alla prima, e costruire intanto il gruppo che rifarebbe ciò che si è appena disfatto:

```
fn apply_inverse<B: BufferTrait>(&mut self, text: &mut B, group: Group) -> Group {
    let mut inverse = Vec::with_capacity(group.changes.len());
    for change in group.changes.iter().rev() {
        match change {
            Change::Inserted { at, len } => {
                // si toglie, ricordando che cosa
                let removed = text.slice(*at, at + len);
                apply_delete(text, *at, at + len);
                inverse.push(Change::Deleted { at: *at, text: removed });
            }
            Change::Deleted { at, text: content } => {
                apply_insert(text, *at, content); // si rimette il testo salvato
                inverse.push(Change::Inserted { at: *at, len: content.chars().count() });
            }
        }
    }
    Group { changes: inverse, point: /* posizione della prima modifica */ }
}
```

`undo` chiude il gruppo aperto, toglie l'ultimo da `done`, lo applica al contrario e mette l'inverso in `undone`; `redo` fa l'operazione simmetrica. Restituiscono la posizione in cui riportare il punto.

- *Il confine è pigro.* Chiamare `boundary` a ogni tasto costava un lock di scrittura sul buffer anche per i tasti che non modificano nulla (circa 120 dei 890 ns di un tasto). Ora un comando timbra un'epoca in una variabile del thread (`CURRENT`), e il gruppo aperto si chiude alla prima modifica che arriva con un'epoca diversa. Le epoche vengono da un contatore globale, così due thread non possono confondere le loro.
- *Digitazione.* `joins_previous` è vero se il comando e il precedente sono entrambi `self-insert`; la serie continua nello stesso gruppo finché il gruppo non ha inserito `AMALGAMATION_LIMIT` (20) caratteri.
- *Macro.* `OneGroup` (RAII, annidabile) blocca il timbro: tutti i comandi di una riproduzione finiscono in un gruppo solo.
- *Fuori dai comandi.* Una modifica fatta da Lisp senza un tasto premuto usa l'epoca rimasta nel thread, quindi si unisce al gruppo dell'ultimo comando; `undo-boundary` chiude il gruppo a mano.
- *Limite.* `DEFAULT_UNDO_LIMIT` è 1 MiB di *testo cancellato* (solo le cancellazioni portano testo); oltre, i gruppi più vecchi vengono scartati. `set-undo-limit` lo cambia per buffer. Il gruppo più recente non viene mai scartato, anche se da solo supera il limite.

8.5 Mark, regione, overlay e testo virtuale

Il mark è una posizione con un flag `active`. La regione esiste solo se il mark è attivo (`region_bounds` la ordina e la limita alla lunghezza del testo), e ogni modifica la disattiva, come in Emacs. Gli *overlay* sono intervalli con una faccia e una categoria (le corrispondenze di una ricerca, una diagnostica, l'enfasi di una pagina di manuale); sono la prima cosa che deve sopravvivere alle modifiche, e le porte li adattano con `adjust_for_insert` e `adjust_for_delete`; un overlay che si riduce a lunghezza zero viene cancellato. Il *testo virtuale* è ancorato a un punto, non a un intervallo, ed è per questo una tabella a parte: il layout delle righe lo inserisce nel testo mostrato (`ui/layout.rs`), e un clic su di esso viene riportato alla posizione vera del buffer.

8.6 Le cache del testo

Cache	Che cosa conserva	Chi la riempie
SyntaxCache	Gli intervalli colorati di ogni riga e lo stato del lessico (la pila delle regioni aperte) all'inizio di ogni riga, per versione.	L'evidenziatore, 500 righe per turno, ogni 40 ms.

Cache	Che cosa conserva	Chi la riempie
ScanCache	Un punto di ripresa ogni 64 righe per la scansione delle espressioni bilanciate, per versione e modo.	Lo scanner preventivo, 500 righe per turno. <code>forward-sexp</code> , l'indentazione e <code>syntax-ppss</code> la leggono e partono dal punto di ripresa più vicino invece che dall'inizio.

Entrambe sono invalidate dalla riga modificata in giù, dalla stessa chiamata (`changed`). Una cache che non corrisponde a versione e modo correnti non viene usata: il caso peggiore è una scansione dall'inizio, cioè il comportamento che c'era prima delle cache.

8.7 File su disco

`find-file` legge il file, imposta `file_path` e il timbro (`FileStamp`: istante di modifica e lunghezza, un solo `stat`), e sceglie il modo secondo le espressioni regolari di `add-auto-mode`. La lettura usa `read_to_string`, che rifiuta un file non UTF-8; il motivo va nel registro ([→ FIL-3](#)). Salvare scrive l'intero testo con `std::fs::write` (sul posto, senza file temporaneo ([→ FIL-2](#))) e aggiorna il timbro, altrimenti ogni salvataggio sembrerebbe la scrittura di qualcun altro. `save-buffer` scrive l'esito, riuscito o fallito, nel registro e non nell'area messaggi ([→ FIL-1](#)) `write-file` lo scrive anche nell'area messaggi.

Il *watcher* (`FileWatcher`, ogni `watch-file-interval` secondi, 3 per default, al massimo 8 file per turno; spento da `(setq watch-files nil)`) confronta i timbri: un buffer pulito il cui file è cambiato viene ricaricato con `adopt_text`; uno modificato, o il cui file è sparito, viene marcato `stale`, e il salvataggio successivo chiede conferma. L'*autosalvataggio* (`AutoSaver`, ogni `auto-save-interval`, 30 secondi per default, al massimo 4 buffer per turno; spento da `(setq auto-save nil)`) scrive i buffer modificati la cui versione è cambiata dall'ultima copia in `#nome#`, nella directory del file o in `auto-save-directory`; `recover-file` riporta la copia nel buffer. La versione salvata viene registrata anche quando la scrittura fallisce ([→ FIL-5](#)).

9 Il testo: ricerca, sostituzione, kill ring, rettangoli e liste di risultati

La directory `core/src/text/` raccoglie ciò che si può calcolare su un testo senza sapere che cosa sia un editor. Il criterio di ammissione è scritto in `text/mod.rs`: un file sta qui se lo potrebbe usare un programma che non è un editor. In pratica nessun file della directory importa `editor` o `lisp`, e nessuno riceve un `Buffer`: ricevono un `BufferTrait` (o una `&str`), cioè qualcosa che sa dare caratteri per posizione, e nient'altro. Niente punto, mark, undo, lock o finestre.

File	Che cosa contiene	Chi lo guida
<code>search.rs</code>	<code>Pattern</code> (che cosa cercare) con <code>search_forward</code> , <code>search_backward</code> , <code>scan</code> ; le sessioni <code>Isearch</code> e <code>Replace</code> ; <code>Casing</code> ed <code>expand_replacement</code> .	<code>feature/isearch.rs</code> , <code>primitives/</code> , <code>isearch.rs</code> <code>editor/replace.rs</code> , <code>primitives/scan.rs</code>
<code>kill_ring.rs</code>	<code>KillRing</code> : un anello di stringhe con la regola di rotazione e di accodamento.	<code>managers/</code> <code>kill_yank.rs</code> (<code>KillYank</code>)
<code>rectangle.rs</code>	<code>Rectangle</code> e <code>Span</code> : l'aritmetica di due angoli e delle righe tra loro.	<code>primitives/</code> <code>rectangle.rs</code>
<code>results.rs</code>	<code>Entry</code> e <code>Results</code> : una lista di posti e un cursore che la percorre.	<code>primitives/</code> <code>results.rs</code>

Le sessioni (`Isearch`, `Replace`) sono qui per la stessa ragione: contengono lo *stato* di un ciclo che l'editor non può scrivere come ciclo, perché non esiste una lettura bloccante di un tasto. Chi le guida tiene la sessione in `Runtime` tra un tasto e l'altro e chiama un metodo per ogni risposta dell'utente; la sessione conosce il buffer solo per nome.

9.1 Posizioni: sempre caratteri

Tutte le posizioni del modulo sono offset in *caratteri*, come il punto, il mark e la storia dell'undo. Il motore delle espressioni regolari (`regex`) lavora invece in *byte* su una `&str`. La conversione avviene in un posto solo: `char_to_byte` all'ingresso e `to_match` (che usa `byte_to_char`) all'uscita. Un errore di conversione non si vede su testo ASCII e rompe il primo carattere accentato, per questo non è ripetuta nei chiamanti.



9.2 Pattern: che cosa cercare

da `text/search.rs` — estratto semplificato

```
pub enum Pattern {
    Literal { chars: Vec<char>, fold: bool }, // confronto carattere per carattere
    Regex(Regex), // compilata una volta sola
}

pub fn new(source: &str, regexp: bool, fold: bool) -> Result<Self, String> {
    if source.is_empty() {
        return Err("Searching for the empty string would never finish".into());
    }
    if regexp {
        RegexBuilder::new(source).case_insensitive(fold).multi_line(true)
            .build().map(Pattern::Regex).map_err(|why| format!("Invalid regexp: {why}"))
    } else {
        Ok(Pattern::Literal { chars: source.chars().collect(), fold })
    }
}
```

Tre decisioni stanno in questa funzione:

- *Il modello vuoto è rifiutato.* Troverebbe una corrispondenza in ogni posizione: una ricerca non si muoverebbe mai e una sostituzione non finirebbe. Rifiutandolo qui nessun chiamante deve ricordarsi il controllo. Un'espressione che *può* corrispondere al vuoto (`x*`) resta invece ammessa, ed è compito di chi cicla scavalcare le corrispondenze vuote (`Match::is_empty`) (→ RIC-1).
- `multi_line(true)`: `^` e `$` indicano inizio e fine di *riga*, come si aspetta chi scrive un'espressione in un editor. Senza, `^`; troverebbe al massimo una corrispondenza in tutto il buffer.
- `fold` arriva dal chiamante, che lo legge dalla variabile Lisp `case-fold-search` (non legata = vero). Per i letterali il confronto ignora le maiuscole con `chars_equal`, che confronta le forme minuscole dei due caratteri.

9.2.1 search_forward e search_backward

`search_forward(text, from, limit)` restituisce la prima corrispondenza che *comincia* in `from` o dopo e *finisce* entro `limit`. Il limite è ciò che confina una sostituzione alla regione: il chiamante passa la fine della regione invece di quella del buffer.

- *Letterale*: prova ogni inizio da `from` a `limit - len` con `matches_at`, che legge i caratteri uno per uno con `BufferTrait::at`. Non alloca nulla: un passaggio, nessuna copia.
- *Espressione*: copia il buffer in una `String` (il motore vuole una `&str`), converte `from` in byte e chiama `captures_at`. Si usa `captures_at` e non una fetta del testo perché la ricerca deve vedere ciò che precede il punto di partenza: altrimenti ogni ripresa sembrerebbe l'inizio di una riga per `^` e per i look-behind. La corrispondenza trovata è scartata se finisce oltre il limite, anche quando ne esisterebbe una più corta che ci sta (→ RIC-4).

da `text/search.rs` — estratto semplificato

```
Pattern::Regex(_) => {
    let mut best = None;
    let mut at = floor;
    while let Some(found) = self.search_forward(text, at, to) {
        at = if found.is_empty() { found.end + 1 } else { found.end };
        best = Some(found);
    }
    best
}
```

`search_backward(text, to, floor)` cerca l'ultima corrispondenza che finisce entro `to` e comincia da `floor` in poi. Per i letterali prova gli inizi all'indietro. Per le espressioni il motore sa cercare solo in avanti,

quindi cammina tutte le corrispondenze da `floor` e tiene l'ultima: ogni passo è una `search_forward`, che ricopia il buffer, e il costo totale cresce con il quadrato del testo (→ RIC-2).

9.2.2 `scan`: tutte le corrispondenze in un passaggio

Occur, `grep` e la lettura dell'output di un compilatore vogliono *tutte* le corrispondenze, con riga, colonna e testo della riga. Chiamare `search_forward` in un ciclo costerebbe una copia del buffer per corrispondenza. `Pattern::scan(haystack, limit)` lavora su una `&str` già pronta e fa un passaggio solo, grazie a un `Cursor` che avanza sempre in avanti e conta caratteri e righe man mano:

da `text/search.rs` — estratto semplificato

```
struct Cursor<'a> {
    haystack: &'a str, byte: usize, chars: usize, line: usize, line_start: usize,
}
fn advance_to(&mut self, byte: usize) {           // byte >= self.byte, sempre
    for (at, c) in self.haystack[self.byte..byte].char_indices() {
        self.chars += 1;
        if c == '\n' { self.line += 1; self.line_start = self.byte + at + c.len_utf8(); }
    }
    self.byte = byte;
}
```

Per ogni corrispondenza (in byte, da `next_match`) `scan` porta il cursore al suo inizio, legge riga e colonna, prende il testo della riga (memorizzato, così venti corrispondenze sulla stessa riga non la ricostruiscono venti volte) e converte inizio e fine in caratteri. Poi riprende dalla fine della corrispondenza, o un carattere dopo se era vuota. Il risultato è una `Scan`: il vettore dei `Found` e `truncated`, vero se si è fermata per `limit`. La colonna invece è ricalcolata contando dall'inizio della riga a ogni corrispondenza (→ RIC-4).

Campo di Found	Significato
<code>start</code> , <code>end</code>	Offset in caratteri nell'intero testo.
<code>line</code>	Riga di inizio, contata da 1 (la numerazione di <code>goto-line</code>).
<code>column</code>	Colonna in caratteri, contata da 0 (come <code>current-column</code>).
<code>line_text</code>	La riga intera senza <code>\n</code> : mostrarla non richiede di rileggere il testo.
<code>groups</code>	Ciò che ha catturato ogni gruppo, 0 = l'intera corrispondenza; vuoto per un letterale.

`in_line()` ed `end_in_line()` danno la posizione della corrispondenza dentro `line_text`, che è ciò che serve a una vista per evidenziarla.

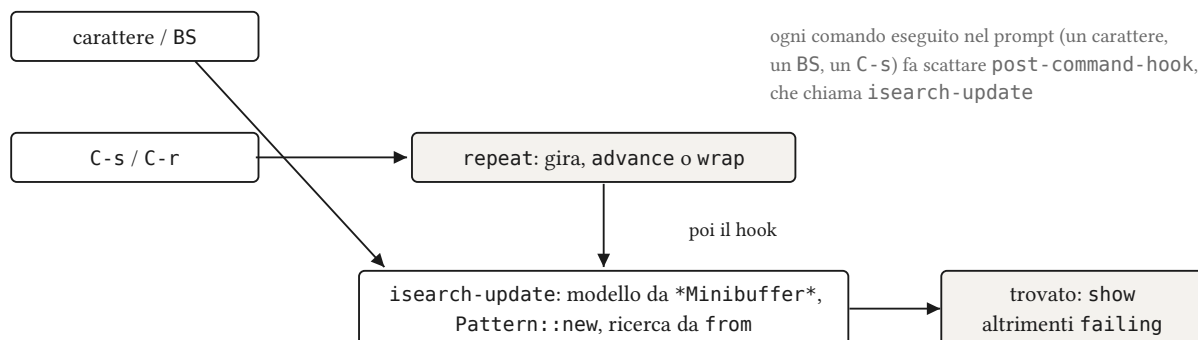
9.3 La sessione di ricerca incrementale: `Isearch`

La parte di interfaccia è descritta nel Sezione 7: `C-s` apre il prompt in `isearch-mode` e il `post-command-hook` di quel modo chiama `isearch-update` dopo ogni comando. Qui interessa lo stato che passa da un tasto all'altro.

da `text/search.rs` — estratto semplificato

```
pub struct Isearch {
    pub buffer: String,           // il buffer cercato (il corrente è il minibuffer)
    pub origin: usize,           // dove era il punto: ripristinato da C-g
    pub direction: Direction,    // Forward | Backward
    pub regexp: bool,
    pub from: usize,             // da dove parte la prossima scansione
    pub found: Option<Match>,    // la corrispondenza mostrata
    pub failing: bool,           // l'ultima scansione non ha trovato nulla
    pub wrapped: bool,           // la ricerca ha fatto il giro
}
```

Il campo chiave è `from`. Scrivere un carattere *non* lo sposta: allungare il modello rifà la ricerca dallo stesso punto, quindi la corrispondenza cresce dove l'utente sta guardando. Solo la ripetizione lo sposta.



Le regole dei metodi, tutte prese dal comportamento di Emacs:

- `advance()` sposta `from` a `found.start + 1` andando avanti (a `found.end - 1` andando indietro), non alla fine della corrispondenza: così anche le corrispondenze sovrapposte sono raggiungibili. Ripetere la ricerca di `aa` in `aaa` trova anche quella che comincia in 1.
- `wrap(len)` riparte dall'altro capo del buffer e imposta `wrapped`. Viene chiamato solo quando si ripete una ricerca che sta già fallendo: il primo C-s a vuoto mostra «Failing I-search», il secondo è la decisione di fare il giro.
- Cambiare direzione non avanza: `from` diventa l'inizio (o la fine) della corrispondenza corrente, perché C-s C-r deve ripercorrere le stesse corrispondenze senza saltarne una.
- `report(pattern)` compone il testo per l'area dei messaggi: `[Failing ][Wrapped ]` seguito da `I-search`, `I-search backward`, `Regexp I-search` o `Regexp I-search backward`, poi `:` e il modello.

`isearch-update` gestisce anche i casi limite. Con il prompt vuoto riporta il punto a `origin` e toglie il mark. Un'espressione che non si compila (`[a-` è a metà di `[a-z]`) è trattata come una ricerca fallita, non come un errore: il carattere successivo la renderà valida. Quando trova, `show` mette il punto sull'estremità lontana della corrispondenza e il mark su quella vicina, così la corrispondenza è la regione e la disegna lo stesso codice che disegna una selezione. `isearch-exit` (Invio) disattiva il mark e lascia il punto dov'è; `isearch-abort` (C-g, ESC) riporta il punto a `origin` (→ RIC-6).

9.4 La sessione di sostituzione: Replace

`replace-string`, `replace-regexp` e `query-replace` creano una `Replace` con `begin_replace` (`editor/replace.rs`). Il campo d'azione è la regione se il mark è attivo, altrimenti dal punto alla fine del buffer: `origin` e `limit` sono i due estremi. Poi `seek` cerca la prima corrispondenza e un overlay della categoria `replace-match` la evidenzia.

da `text/search.rs` — estratto semplificato

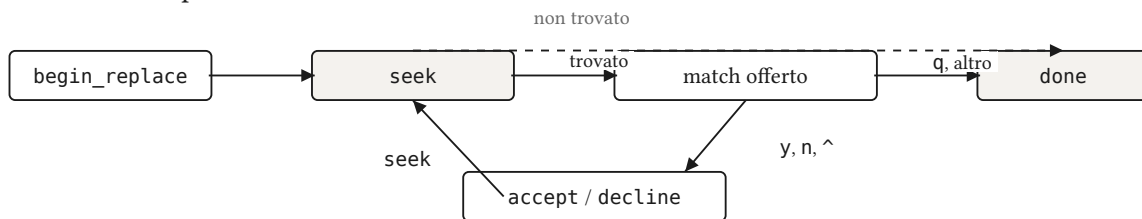
```

pub struct Replace {
    pub buffer: String, pub pattern: Pattern,
    pub replacement: String, // il modello, con \1 \& ancora dentro
    pub origin: usize, pub from: usize,
    pub limit: usize, // spostato da ogni sostituzione
    pub found: Option<Match>, pub replaced: usize,
    pub history: Vec<Step>, // da dove partiva ogni sostituzione accettata
    pub preserve_case: bool, pub done: bool,
}
  
```

Chi la guida chiama un verbo per ogni risposta. La vista predefinita (`default_view` in `primitives/replace.rs`) installa una keymap transitoria con cinque tasti: `y` (`replace-this`), `n` (`replace-skip`), `!` (`replace-rest`), `^` (`replace-back`, che usa `unaccept` e l'undo del buffer) e `q` (`replace-done`). La keymap è di tipo `OnUnbound::Release`: qualunque altro tasto chiude la sostituzione e poi fa ciò che fa di solito. Un'interfaccia grafica userebbe pulsanti; nessuna delle due si esprime come un ciclo che legge una risposta.

Verbo	Effetto sulla sessione
<code>seek(text)</code>	<code>found = search_forward(text, from, limit)</code> ; se non trova nulla, <code>done = true</code> .
<code>accept(new_len)</code>	La corrispondenza è stata sostituita da <code>new_len</code> caratteri. Registra <code>Step { from: start, limit }</code> in <code>history</code> ; sposta <code>limit</code> della differenza di lunghezza; riprende da <code>start + new_len</code> (o <code>start + 1</code> se corrispondenza e sostituzione erano entrambe vuote) ($\rightarrow$ RIC-1). Incrementa <code>replaced</code> .
<code>decline()</code>	Salta la corrispondenza: riprende dalla sua fine, o un carattere dopo se era vuota.
<code>unaccept()</code>	Riprende l'ultimo <code>Step</code> : <code>from</code> e <code>limit</code> tornano com'erano, <code>replaced</code> cala. Il testo lo rimette a posto il chiamante con l'undo del buffer, perché la sessione non sa nulla dei buffer.

Il limite si sposta perché una sostituzione più lunga o più corta sposta tutto ciò che segue, compresa la fine della regione. Senza questo aggiornamento una sostituzione confinata alla regione ne uscirebbe, o si fermerebbe prima. La `history` serve a tornare indietro: dopo una sostituzione di lunghezza diversa il testo da solo non dice più dove era cominciata.



Il testo da inserire lo calcola, per ogni corrispondenza, `expand_replacement`, che riceve il modello di sostituzione, la corrispondenza, il testo effettivamente trovato e `preserve_case`:

- `\1 ... \9` sono i gruppi catturati; un gruppo che non ha partecipato, o qualsiasi gruppo con un modello letterale (che non ne ha), diventa la stringa vuota;
- `\0` e `&` sono l'intera corrispondenza, presa dal testo (`matched`) e non dal gruppo 0, così funzionano anche con i letterali;
- `\` è una barra rovesciata; una barra seguita da qualunque altra cosa resta com'è, così un percorso Windows si può scrivere senza raddoppiare le barre.

Con `preserve_case` (il valore di `case-replace` quando la ricerca ignora le maiuscole) il risultato viene poi riscritto secondo `Casing::of(matched)`:

Corrispondenza	Casing	foo diventa
BAR	Upper: tutte le lettere maiuscole (almeno due)	F00
Bar, B	Capitalised: la prima maiuscola, le altre minuscole	Foo
bar, bAr, fooBar, 42	AsWritten: minuscole, miste o nessuna lettera	foo

Sul lato dell'editor, `replace_this` cancella la corrispondenza e inserisce l'espansione con le due porte (`delete_range`, `insert_text`), chiude un gruppo di undo ($\rightarrow$ RIC-3), chiama `accept` e poi cerca la

successiva. `replace_rest` (il ! di `query-replace`, e tutto `replace-string`) ripete `replace_this` con un tetto di `len + 1` giri, che è l'unica cosa che ferma il caso delle corrispondenze vuote (→ RIC-1).

9.5 Il kill ring

da `text/kill_ring.rs` — estratto semplificato

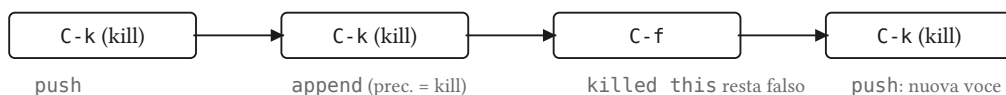
```
pub struct KillRing {
    entries: VecDeque<String>, // la più recente in testa
    max: usize,                 // DEFAULT_KILL_RING_MAX = 60
    index: usize,               // da dove legge yank; azzerato da ogni kill
}

pub fn append(&mut self, text: String, direction: Direction) {
    if text.is_empty() { return; }
    match self.entries.front_mut() {
        Some(front) => {
            match direction {
                Direction::Forward => front.push_str(&text),
                Direction::Backward => front.insert_str(0, &text),
            }
            self.index = 0;
        }
        None => self.push(text),
    }
}
```

Un anello invece di un appunto singolo: uccidere del testo non distrugge quello che si stava per incollare. Le operazioni:

- `push(text)` aggiunge una voce in testa e azzerava `index`; il testo vuoto è ignorato, così una kill di niente non allontana ciò che si voleva incollare; poi `trim` scarta le voci oltre `max`.
- `append(text, direction)` unisce il testo alla voce più recente: in coda per una kill in avanti, in testa per una all'indietro. Una serie di C-k diventa così un'unica voce che si legge nell'ordine giusto, e una serie di M-DEL non viene assemblata al contrario.
- `current()` è ciò che inserisce `yank`; `rotate()` avanza `index` di uno (con giro) ed è ciò che usa `yank-pop`; `nth(n)` legge senza muovere l'anello.
- `set_max (set-kill-ring-max)` riduce subito l'anello; se `index` finisce oltre la fine torna a 0.

La decisione tra `push` e `append` non è dell'anello ma del compartimento `KillYank` (`managers/kill_yank.rs`), che tiene due coppie di flag: `killed_last / killed_this` e `yanked_last / yanked_this`. Una kill accoda se anche il comando precedente era una kill; `roll_over`, chiamato all'inizio di ogni comando, sposta «questo» in «precedente» e azzerava «questo». Allo stesso modo `yank-pop` è permesso solo se il comando precedente era uno yank, e `last_yank` dice quale tratto di testo sostituire.



tra un comando e l'altro `roll_over`: `killed_last = killed_this`; `killed_this = false`

Se la variabile `clipboard-sync` è vera (la imposta `clipboard.lisp`), ogni kill lascia anche la voce corrente in `pending_clipboard`. La `snapshot` successiva la prende (`take_pending_clipboard`) e il TUI la manda al terminale con una sequenza OSC 52. Prenderla invece di leggerla evita di rimandarla a ogni ridisegno.

L'ultimo rettangolo ucciso o copiato non va nell'anello: sta in un campo a parte di `KillYank`, una riga per stringa, e `yank-rectangle` lo legge da lì.

9.6 I rettangoli

Il punto e il mark sono due angoli; il rettangolo è il blocco di colonne che le righe tra loro hanno in comune. Non è la regione, che dalla metà di una riga alla metà di un'altra prende anche le righe intere in mezzo.

da `text/rectangle.rs` – estratto semplificato

```
pub struct Rectangle { pub top: usize, pub bottom: usize, pub left: usize, pub right: usize }
pub fn between(a: (usize, usize), b: (usize, usize)) -> Self { // (riga, colonna)
    Self { top: a.0.min(b.0), bottom: a.0.max(b.0), left: a.1.min(b.1), right: a.1.max(b.1) }
}
```

`bottom` è incluso e `right` no: le righe sono un insieme di cose, le colonne un intervallo di posizioni. Un rettangolo largo zero è ammesso ed è utile: è una posizione su ogni riga, che è ciò che serve a `string-rectangle` per anteporre un prefisso a un gruppo di righe. Gli angoli possono essere dati in qualsiasi ordine.

Le colonne sono caratteri. `rsedit` non espande le tabulazioni (ne disegna una per cella), quindi colonna di carattere e colonna sullo schermo coincidono (→ TUI-3).

`spans(text, rect)` traduce il rettangolo in un `Span` per riga, dall'alto:

da `text/rectangle.rs` – estratto semplificato

```
(rect.top..=rect.bottom.min(last_line)).map(|line| {
    let line_start = text.cursor_2d_to_1d(line, 0);
    let start = text.cursor_2d_to_1d(line, rect.left); // limitato alla fine della riga
    let end = text.cursor_2d_to_1d(line, rect.right);
    let reached = start - line_start;
    Span { line, start, end, padding: rect.left - reached.min(rect.left) }
})
```

Tutto si regge sul fatto che `cursor_2d_to_1d` limita una colonna oltre la fine della riga alla fine della riga. Su una riga corta entrambi gli estremi cadono sulla fine e lo span è vuoto; `padding` dice quanti spazi mancano per raggiungere il bordo sinistro. Esempio con `left = 4`, `right = 7`:

Riga	Span	padding	Testo coperto
0123456789	4..7	0	456
01234	4..5	0	4 (la riga finisce prima del bordo destro)
01	2..2	2	vuoto
(riga vuota)	0..0	4	vuoto

Che cosa fare delle righe corte lo decide l'operazione. Togliere testo (`kill-rectangle`, `copy-rectangle-as-kill`, `clear-rectangle`) prende ciò che c'è e non allunga la riga. Mettere testo (`open-rectangle`, `yank-rectangle`, `string-rectangle`) prima aggiunge il padding, perché una colonna che non esiste è proprio ciò che quelle operazioni devono riempire. Gli span sono dall'alto perché è l'ordine di lettura; chi modifica li percorre dal basso, perché una modifica a una riga sposta tutti gli offset successivi. `text_of` restituisce il testo di ogni span, con una stringa vuota per le righe che non raggiungono il rettangolo, così il blocco mantiene la sua forma.

9.7 Le liste di risultati

Occur, `grep` e la lettura dell'output di un compilatore producono la stessa cosa: una lista di posti dove andare. `results.rs` la definisce una volta, così produrre la lista e percorrerla sono lavori separati.

Campo di Entry	Significato
kind	"buffer" o "file": altrimenti <i>*scratch*</i> e un percorso si distinguerebbero solo indovinando.
source	Nome del buffer o percorso del file.
line, column	Riga da 1, colonna da 0.
offset	Offset in caratteri, per goto-char.
text, start, end	La riga intera e la posizione della corrispondenza al suo interno.

Entry::new(kind, source, found) copia i campi da un Found di scan; rendered() produce la forma file:riga: testo di ogni strumento alla grep. Results aggiunge il modello cercato, su che cosa (over), le voci, truncated, done (falso mentre una scansione di sfondo è in corso), il buffer dove è stato messo l'ultimo evidenziatore (marked, per toglierlo) e la voce corrente.

da text/results.rs — estratto semplificato

```
pub fn step(&self, step: isize) -> Option<usize> {
    if self.entries.is_empty() { return None; }
    let next = match self.current {
        None if step >= 0 => 0,           // prima visita in avanti: la prima
        None => self.entries.len() - 1,   // prima visita indietro: l'ultima
        Some(current) => {
            let next = current as isize + step;
            if next < 0 || next as usize >= self.entries.len() { return None; }
            next as usize
        }
    };
    Some(next)
}
```

Il passo non fa il giro, di proposito: la fine della lista è un'informazione («era l'ultimo»), e ricominciare in silenzio farebbe correggere due volte il primo errore. Un insieme di risultati vive nel buffer che lo mostra, in Buffer::data sotto la chiave "results" (RESULTS_KEY): quando il buffer viene chiuso i risultati spariscono con lui, e rinominarlo non rompe nulla. Le voci conservano numeri di riga e offset, non marcatori che seguono il testo (→ RIC-5).

10 Modi

10.1 Il modo maggiore

Un buffer ha un solo modo maggiore, nominato in `current_mode`. Il modo è un `MajorMode` nel registro di `Modes`:

Campo	Contenuto
<code>keymaps</code>	Le associazioni del modo, consultate prima di quelle globali.
<code>grammar</code>	Regole e regioni di colorazione (<code>add-syntax-rule</code> , <code>add-syntax-region</code>).
<code>hooks</code>	Hook per nome (<code>post-command-hook</code> , <code>post-self-insert-hook</code> , <code>after-close-hook</code> , ...), aggiunti con (<code>add-hook</code> <code>MODO</code> <code>NOME</code> <code>FUNZIONE</code>).
<code>completion_functions</code>	Le fonti di <code>completion-at-point</code> per questo modo.
<code>syntax_table</code>	Delimitatori, stringhe, commenti, quote dei caratteri: ciò che legge lo scanner delle espressioni.

`Modes` contiene inoltre la keymap globale, i completamenti globali, la lista `auto_modes` (espressione regolare sul nome del file → modo, da `add-auto-mode`), la keymap transitoria, i tasti di ripetizione e gli hook globali, che valgono per ogni modo e si concatenano a quelli del modo.

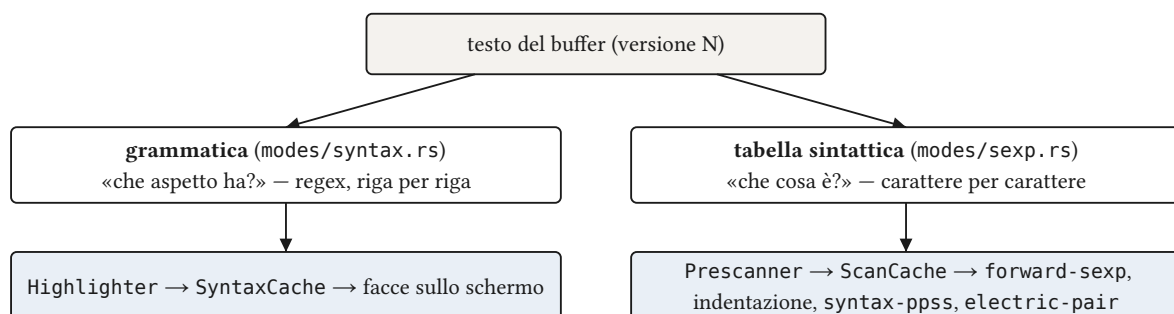
`make-mode` registra un `MajorMode` nuovo sotto il nome dato, anche se il nome esiste già: keymap, grammatica e hook definiti prima per quel nome vengono sostituiti (→ [SIN-4](#)).

Le proprietà dei simboli completano il modo senza toccare la struttura Rust; queste sono lette da `indent.lisp`:

```
(put 'rust-mode 'tab-width 4)
(put 'risp-mode 'indent-function 'lisp-indent-line)
(put 'rust-mode 'indent-region-style 'shift)
```

10.2 Due lessici sullo stesso testo

Il testo di un buffer viene letto in due modi diversi, per due domande diverse.



La *grammatica* colora una riga alla volta portando uno stato da una riga all'altra: lo stato è la pila delle regioni aperte (un commento a blocchi che continua, una stringa su più righe). Colorare la riga 5000 richiede solo lo stato all'inizio della riga 5000, che la cache conserva; dopo una modifica si ricolorela dalla riga modificata in giù. In ogni riga tutti i modelli attivi sono provati insieme e *vince la corrispondenza più a sinistra* a parità di posizione le regioni vengono prima delle regole. Così `// this /* is not` è un commento di riga e non apre un commento a blocchi. Le espressioni regolari sono quelle della crate `regex`, senza lookahead,

lookbehind né backreference: per questo una regola può colorare solo un gruppo (`SyntaxRule::group`) e una regione può dichiarare una sequenza di escape.

La *tabella sintattica* dice quali caratteri aprono e chiudono liste, quali delimitano stringhe (`set-string-syntax`, anche multi-carattere come `r#"`), quali commenti (`set-comment-syntax`) e quale carattere quota un carattere (`set-char-quote`). Le due descrizioni sono indipendenti: aggiungere una forma di stringa a un modo significa aggiungerla in entrambe.

10.3 Lo scanner delle espressioni

`modes/sexp.rs` risponde alle domande sulla struttura del testo: dove finisce l'espressione dopo il punto (`forward-sexp`), dove comincia quella prima (`backward-sexp`), in quale lista, stringa o commento si trova una posizione (`syntax-ppss`, che l'indentazione e `electric-pair` usano). Contare le parentesi non basta: in `(message "close it with ")` tre parentesi su quattro sono testo. Lo scanner quindi fa un'analisi lessicale con quattro stati, e conta un delimitatore solo quando lo sta leggendo come codice. Gli stati e le transizioni:

Stato	Ci si entra da Code quando...	Si torna a Code quando...
Code	— (stato iniziale)	—
Str { quote, start }	si legge un carattere di classe <code>StringQuote</code> .	si rilegge la stessa virgoletta; un carattere <code>Escape</code> fa saltare anche il successivo.
StyledStr { style, start }	comincia un'apertura multi-carattere dichiarata con <code>set-string-syntax</code> (<code>r#"</code> , <code>"""</code>).	si legge la chiusura dello stile. Nessun escape: in una stringa grezza <code>\</code> è solo <code>\</code> .
Comment { style, depth, start }	comincia un'apertura di commento di <code>set-comment-syntax</code> (vince la più lunga).	riga: al <code>\n</code> . Blocco: alla chiusura con <code>depth 0</code> ; se lo stile è <code>nestable</code> un'altra apertura incrementa <code>depth</code> .

Lo stato completo di una scansione è lo stato lessicale più una pila di `Frame`, uno per ogni lista aperta. Il fondo della pila è il buffer stesso e non viene mai tolto: una chiusura in più non svuota mai la pila.

da `modes/sexp.rs` — estratto semplificato

```
struct Frame {
    start: usize,           // inizio dell'espressione, prefisso incluso
    delimiter: usize,       // dove sta la parentesi
    children: VecDeque<Sexp>, // le ultime `keep` espressioni completate qui
}
pub struct Scan<'a, B: BufferTrait> {
    text: &'a B, table: &'a SyntaxTable,
    pos: usize, state: State, stack: Vec<Frame>,
    prefix: Option<usize>, // un carattere prefisso (' ` # ,) in attesa
    keep: usize,
}
```

`step()` fa un passo e dice se ha completato un'espressione (`Completed(depth, Sexp)`), se ha solo consumato caratteri (`Advanced`) o se il testo è finito (`End`). In stato `Code` l'ordine dei controlli conta:

1. Un'apertura di commento (la più lunga che corrisponde) entra in `Comment` e scarta un prefisso in attesa. Anche un commento completato conta come espressione, così `forward-sexp` lo scavalca.
2. Un'apertura di stringa multi-carattere viene prima dei caratteri singoli, perché contiene virgolette che altrimenti aprirebbero una stringa normale.
3. Un letterale di carattere (`'a'`, `'''`) è un'unica espressione. Il `'` non ha una classe fissa: apre un carattere in `'a'`, una lifetime in `'static`, un apostrofo nella prosa; lo decide `SyntaxTable::char_literal_at` guardando ciò che segue.

4. Poi la classe del carattere: `Open` mette un `Frame` sulla pila; `Close` toglie quello in cima e completa la lista. Una chiusura che non corrisponde all'apertura chiude lo stesso, e una chiusura senza niente di aperto è testo: un file è sbilanciato per tutto il tempo in cui lo si scrive, e uno scanner che si fermasse lì sarebbe inutile proprio quando serve. `StringQuote` entra in `Str`; `Escape` salta due caratteri; `Prefix` ricorda la posizione del primo prefisso di una serie (`#'foo` comincia al `#`); `Symbol` consuma l'intera serie di costituenti e completa un'espressione; `Punctuation` avanza, e uno spazio stacca un prefisso in attesa.

Ogni espressione completata entra nei `children` del frame in cima, che ne tiene al massimo `keep`. È il limite che rende la scansione economica: `backward-sexp` con un conteggio `N` ha bisogno solo delle ultime `N`.

Funzione	Come usa la scansione
<code>forward(from, n)</code>	Scansione fino a <code>from</code> , poi passi finché non si completano <code>n</code> espressioni <i>allo stesso livello</i> che finiscono dopo <code>from</code> . Se la lista finisce prima, il punto non si muove: alla fine di una lista il comando non esce verso l'esterno ($\rightarrow$ SIN-6).
<code>backward(from, n)</code>	Scansione fino a <code>from</code> con <code>keep = n</code> ; il frame rimasto aperto è la lista in cui si trova il punto, e i suoi <code>children</code> sono i fratelli prima del punto. Restituisce l'inizio dell' <code>N</code> -esimo dal fondo, prefisso incluso.
<code>down(from)</code>	Scansione finché la profondità non supera quella di partenza: il punto è appena entrato in una lista.
<code>context_at(pos)</code>	Scansione fino a <code>pos</code> , poi <code>context()</code> : profondità, inizio e delimitatore della lista più interna, inizio della stringa o del commento in corso. È ciò che restituisce <code>syntax-ppss</code> .
<code>balance_point(pos)</code>	La prima posizione dopo <code>pos</code> in cui tutte le liste aperte in <code>pos</code> sono chiuse. Non chiede se il buffer è bilanciato alla fine: una funzione chiusa resta chiusa anche se quella sotto è ancora da finire.
<code>enclosing(pos)</code>	La lista più interna che contiene <code>pos</code> , per intero (fino alla fine del testo se non è chiusa); <code>None</code> al livello zero.

10.3.1 Riprendere da metà: `Resume` e il `Prescanner`

Andare all'indietro non si può fare localmente: se una `)` prima del punto sia codice o testo dipende da tutto ciò che la precede, perché una stringa aperta duecento righe sopra la rende testo. Quindi ogni scansione va in avanti, e può cominciare solo in una posizione di cui si conosce lo stato. All'inizio quella posizione era solo la 0; ora `Scan::begin` accetta un `Resume`, una fotografia dello stato presa prima:

da `modes/sexp.rs` — estratto semplificato

```
pub struct Resume { pos: usize, state: State, stack: Vec<Frame>, prefix: Option<usize> }
pub fn begin(text, table, before: usize, resume: Option<&Resume>, keep: usize) -> Self {
    if let Some(from) = resume.filter(|from| from.pos <= before) {
        return Self { pos: from.pos, state: from.state.clone(),
                      stack: from.stack.clone(), .. }; // riprende da lì
    }
    Self { pos: 0, state: State::Code,
          stack: vec![Frame { start: 0, delimiter: 0, .. }], .. } // dall'inizio
}
```

Il controllo `from.pos <= before` è l'intero argomento di sicurezza: una fotografia è solo una scorciatoia per arrivare a una posizione, mai un sostituto. Una fotografia sbagliata o troppo avanti costa una scansione completa, non una risposta sbagliata. Le fotografie sono prese senza i `children` dei frame, che dipendono da `keep`: per questo `backward` non riprende mai e scansiona sempre dall'inizio del buffer.

Le fotografie le produce il `Prescanner`, un lavoro di sfondo come l'evidenziatore (Sezione 11). Ne prende una ogni `LINES_PER_CHECKPOINT` (64) righe, all'inizio delle righe 64, 128, 192... (`checkpoint_line`), e le salva nella `ScanCache` del buffer insieme alla versione del testo e al nome del modo. Una fotografia presa con la tabella di un altro modo sarebbe una risposta sbagliata che sembra giusta, e per questo il modo è controllato. Dopo una modifica alla riga `L`, `invalidate_from` tiene le fotografie delle righe fino a `L` (lo stato all'ingresso di una riga dipende solo dal testo prima) e scarta le altre; `record` accetta solo la fotografia successiva all'ultima valida. `Buffer::scan_resume(pos)` sceglie quella della riga multipla di 64 più vicina sotto `pos`, se versione e modo corrispondono.

10.4 Facce e temi

Una faccia è un `Face(u16)`: un indice in un registro globale. Le prime 14 sono predefinite (`default`, `region`, `mode-line`, `mode-line-inactive`, `window-separator`, `keyword`, `type`, `string`, `comment`, `function`, `builtin`, `line-number`, `line-number-current`, `replace-match`); un modulo crea una faccia nuova semplicemente nominandola (`'doc-comment` in `rust-mode.lisp`) e le dà un aspetto con `set-face`. Il `Theme` in `Runtime` associa a ogni faccia uno `Style` (colori di primo piano e sfondo, grassetto, corsivo, sottolineato, inverso). I colori sono richieste — un nome o `#rrggbb` — e il front-end decide come approssimarli (Sezione 12).

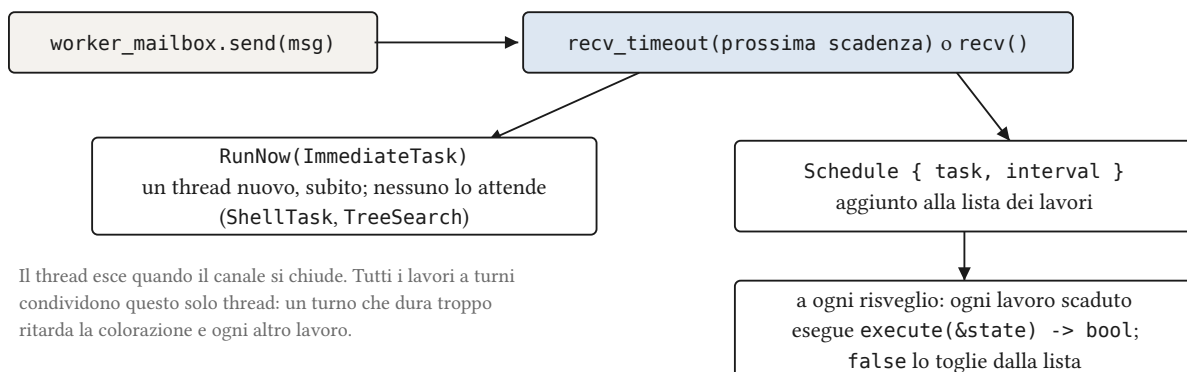
11 Il lavoro in background

11.1 La regola

`background/worker.rs` enuncia sette regole per tutto ciò che gira fuori dal thread dei comandi. In breve: ciò che può durare più di un frame è un lavoro di sfondo avviato da una sola chiamata; un lavoro non disegna mai; non presume che l'editor sia rimasto fermo e riconrolla prima di salvare; il progresso è un segnale, non un ridisegno; le callback girano sul thread dei comandi, tra un comando e l'altro; gli errori vanno nel log e il lavoro dichiara comunque di aver finito; un lavoro con lo stesso nome di uno in corso lo sostituisce.

11.2 Lo scheduler

`BackgroundScheduler::spawn` avvia un thread che riceve `WorkerMessage` dalla mailbox dell'editor.



da `background/mod.rs` — estratto semplificato

```
std::thread::spawn(move || {
    let mut scheduled_jobs: Vec<Job<B>> = Vec::new();
    loop {
        let now = Instant::now();
        let time_to_next_job = scheduled_jobs.iter()
            .map(|job| job.next_run.saturating_duration_since(now)).min();
        let msg = match time_to_next_job {
            // si dorme fino alla prossima scadenza
            Some(timeout) => receiver.recv_timeout(timeout),
            None => receiver.recv().map_err(|_| RecvTimeoutError::Disconnected),
        };
        match msg {
            Ok(WorkerMessage::RunNow(task)) => { /* thread::spawn: task.execute(&state) */ }
            Ok(WorkerMessage::Schedule { task, interval }) =>
                scheduled_jobs.push(Job {
                    task, interval, next_run: Instant::now() + interval,
                }),
            Err(RecvTimeoutError::Disconnected) => break, // l'editor è stato distrutto
            Err(RecvTimeoutError::Timeout) => {}
        }
        let current_time = Instant::now();
        let _off_the_command_thread = WorkerTurn::begin();
        scheduled_jobs.retain_mut(|job| {
            if job.next_run > current_time { return true; }
            let keep_running = job.task.execute(&state); // false: il lavoro è finito
            if keep_running { job.next_run = current_time + job.interval; }
            keep_running
        });
    }
});
```

Il thread non ha un ciclo di attesa attiva: dorme in `recv_timeout` fino alla scadenza più vicina, o in `recv` se non ha lavori. Un messaggio lo sveglia in anticipo. Ogni turno di ogni lavoro gira sullo stesso thread, uno dopo l'altro, e nessuno lo protegge da un panic: un turno che va in panic termina il thread e con lui tutti i lavori a turni (→ BKG-1).

Lavoro	Tipo, intervallo	Che cosa fa a ogni turno
Highlighter	a turni, 40 ms	Per ogni buffer con colorazione da fare: copia fino a 500 righe sotto lock, rilascia, colora, riprende il lock e salva solo se la versione non è cambiata.
Prescanner	a turni, 40 ms	Lo stesso per la <code>ScanCache</code> : 500 righe, un punto di ripresa ogni 64. Esamina le righe sotto il lock di lettura.
FileWatcher	a turni, 3 s	Confronta i timbri di al massimo 8 file; ricarica o marca <code>stale</code> .
AutoSaver	a turni, 30 s	Scriva al massimo 4 buffer modificati e cambiati dall'ultima copia.
worker Lisp	a turni, 40 ms o <code>INTERVAL</code>	<code>resume</code> della fiber, con <code>worker-fuel</code> passi (200.000 per default).
background-call	a turni o una volta	Una fiber a turni, o una funzione eseguita una volta sul thread dello scheduler.
ShellTask	thread proprio	Esegue un processo e accoda l'uscita al buffer.
TreeSearch	thread proprio	Cerca nei file di una directory e scrive i risultati in un buffer.

11.3 Un turno dell'evidenziatore

L'evidenziatore è il lavoro di sfondo più frequente, e il modello per scriverne altri. Un turno si divide in tre fasi, e nessuna tiene un lock mentre fa il lavoro vero:

da `modes/highlighter.rs` — estratto semplificato

```
pub(crate) fn highlight_one_turn(&self) {
    let Some(turn) = self.next_highlight_turn() else { return }; // 1. sotto lock: copia
    let coloured = turn.run(); // 2. senza lock: colora
    self.store_turn(&turn, coloured); // 3. sotto lock: salva
}
pub(crate) fn store_turn(
    &self, turn: &Turn, coloured: Vec<(usize, SyntaxState, Vec<SyntaxSpan>)>,
) {
    self.with_buffer_mut(&turn.buffer, |buf| {
        if buf.version != turn.version { return; } // il testo è cambiato: si butta tutto
        for (line, entering, spans) in coloured {
            if buf.syntax.record(line, &entering, spans).is_err() { break; }
        }
    });
}
```

1. *Copia.* `next_highlight_turn` sceglie il primo buffer, partendo da quello con il focus, la cui cache è indietro rispetto al testo (`syntax.valid_to() < line_count()`) e il cui modo ha una grammatica. Copia fino a `LINES_PER_TURN` (500) righe a partire dalla prima non valida, lo stato del lessico all'inizio di quella riga, la grammatica (clonata dal registro dei modi) e la versione del buffer.
2. *Colora.* `Turn::run` chiama `highlight_line` su ogni riga, passando lo stato in uscita da una riga come stato in entrata della successiva. Il costo di una riga cresce più che linearmente con la sua lunghezza (→ SIN-2).
3. *Salva.* Solo se la versione del buffer è ancora quella copiata: un carattere digitato nel frattempo rende inutile tutto il turno, che viene rifatto al giro successivo dalla riga modificata.

Lo stesso schema — copia sotto lock, lavora senza, ricontrolla la versione prima di salvare — regge il `Prescanner` (che però esamina le righe sotto il lock di lettura, perché la scansione è veloce) e l' `AutoSaver`.

11.4 I comandi di shell

`shell-command-start` avvia `/bin/sh -c COMANDO` (o `cmd /C` su Windows) con stdin chiuso e stdout e stderr in pipe, crea un buffer `*Shell Output*` (o `<2>`, `<3>`... se il nome è occupato) e spedisce allo scheduler un `RunNow(ShellTask)`. Il task gira su un thread suo:

da `primitives/shell.rs` — estratto semplificato

```
fn execute(mut self: Box<Self>, state: &EditorState<B>) {
    if let Some(stdout) = self.child.stdout.take() {
        for line in BufReader::new(stdout).lines() { // riga per riga, man mano
            match line {
                Ok(line) => { state.append_to_buffer(&self.buffer, &format!("{line}\n")); }
                Err(why) => { /* "[unreadable output: ...]" */ break; }
            }
        }
    }
    if let Some(mut stderr) = self.child.stderr.take() { // poi stderr, tutto insieme
        let mut text = String::new();
        if stderr.read_to_string(&mut text).is_ok() && !text.is_empty() {
            state.append_to_buffer(&self.buffer, &text);
        }
    }
    let status = /* "--- exited N ---" o "--- killed ---" */;
    state.append_to_buffer(&self.buffer, &status);
    state.finish_shell_command(); // il ridisegno periodico può fermarsi
}
```

`append_to_buffer` toglie per un attimo la sola lettura del buffer e scrive con `insert_text`, quindi l'output passa dalle porte come ogni altra modifica e il renderer lo vede al primo ridisegno (finché un comando è in corso `next_redraw_in` chiede un ridisegno ogni 40 ms). Lo stdout è letto per intero prima di cominciare a leggere lo stderr (→ SHL-1) la prima riga non UTF-8 interrompe la lettura (→ SHL-2) non esiste un modo di fermare il processo (→ SHL-3). `shell-command-to-string`, al contrario, è sincrono: aspetta la fine del processo sul thread dei comandi e restituisce lo stdout come stringa.

11.5 Worker e lavori in Lisp

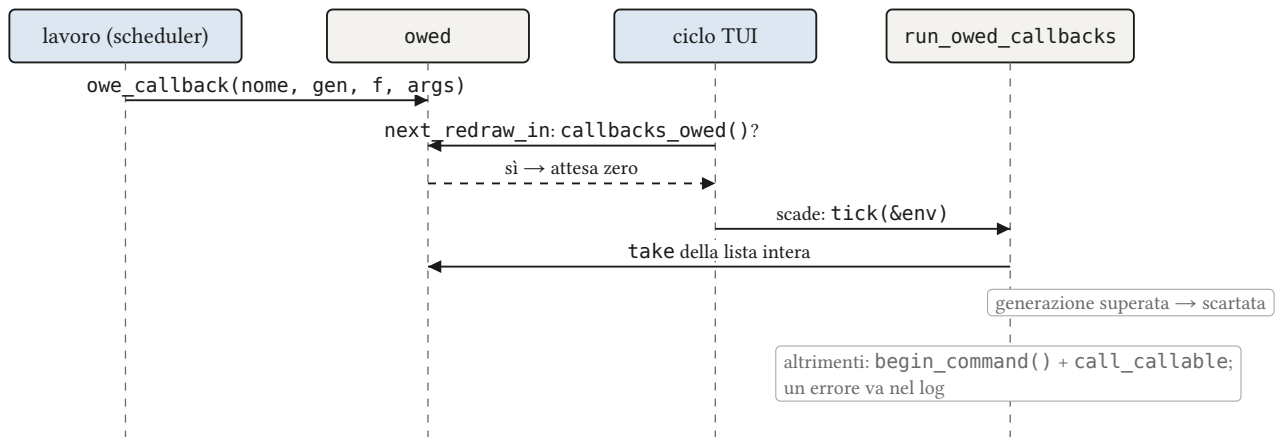
`(define-worker NOME FIBER [INTERVALLO])` riprende la fiber a ogni turno; la fiber è un ciclo che fa `(yield)` dove accetta di essere messa da parte. Ogni turno ha un suo budget, impostato sul thread dello scheduler con `set_remaining(worker-fuel)`: il carburante è per thread, quindi un comando non può accorciare il turno di un worker né un worker spendere quello di un comando. Un turno che supera il budget termina, viene segnalato una volta e il worker viene ritirato: un `(while t)` senza `yield` fermerebbe altrimenti la colorazione per sempre.

`(background-call NOME LAVORO [ON-PROGRESS [ON-DONE [INTERVALLO]]])` avvia un lavoro e restituisce subito. `ON-PROGRESS` è chiamata con il nome dopo ogni turno tranne l'ultimo; `ON-DONE` una volta, con il nome e `t` (finito) o `nil` (errore o arresto). `stop-worker` e `running-workers` valgono per entrambi.

Ogni nome ha una *generazione* in `Runtime`: definire di nuovo lo stesso nome, o fermarlo, incrementa il numero, e il lavoro vecchio si ritira al prossimo risveglio confrontando la generazione con cui è nato. Non serve un canale di cancellazione, e ricaricare un modulo non duplica i suoi worker.

11.6 Le callback dovute

Una callback chiesta da un lavoro di sfondo non gira sul thread del lavoro: va nella lista `owed` dell'editor, con il nome e la generazione del lavoro.



da `editor/background.rs` – estratto semplificato

```

pub fn run_owed_callbacks(&self, env: &Arc<Env<EditorState<B>>>)) -> bool {
    let owed = std::mem::take(&mut *self.owed.write().unwrap()); // la lista intera
    let mut ran = false;
    for callback in owed {
        if !self.runtime(|r| r.worker_is_latest(&callback.name, callback.generation)) {
            continue; // il lavoro è stato sostituito o fermato
        }
        ran = true;
        let _command = self.begin_command(); // budget pieno
        if let Err(error) =
            call_callable(&callback.function, &callback.args, env.clone(), self)
        {
            self.log_diagnostic(/* la callback del lavoro è fallita */);
        }
    }
    ran
}

```

La lista viene svuotata con un solo `take` prima di eseguire qualunque callback: una callback che chiede altre callback le aggiunge a una lista nuova, che girerà al prossimo `tick`, e il lock non resta mai preso mentre gira Lisp.

È qui che un modo «resta coerente» (regola 5): la callback gira tra un comando e l'altro, sul thread che possiede l'interfaccia, e può aprire buffer, scrivere messaggi o cambiare la finestra come farebbe un comando.

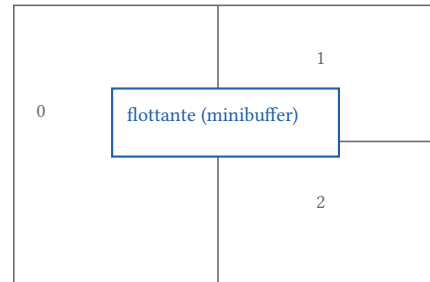
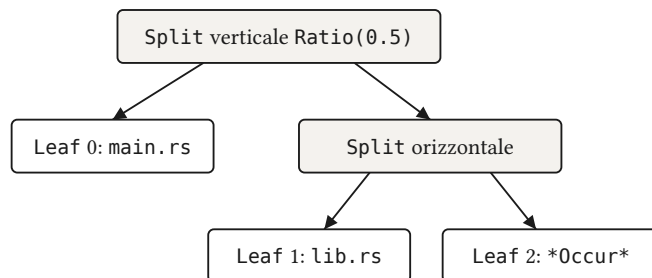
11.7 Quando ridisegnare

`next_redraw_in(&env, &frame)` è il minimo tra: la scadenza del messaggio nell'area messaggi (`echo-message-timeout`, 5 s; `nil` non scade mai); 40 ms se la colorazione è in corso, se c'è un comando di shell attivo o se un lavoro di sfondo sta cambiando lo schermo; zero se ci sono callback dovute. `None` significa che nulla è in sospeso e il front-end può bloccarsi sull'input. `drag_scroll_in()` aggiunge i 60 ms dello scorrimento automatico mentre si trascina una selezione oltre il bordo di una finestra.

12 Lo schermo

12.1 Finestre

Le finestre affiancate formano un albero (`LayoutNode`): una foglia è una `Window` (id, nome del buffer, scorrimento orizzontale e verticale, punto proprio quando non ha il focus, larghezza del margine), un nodo interno è una divisione orizzontale o verticale con una `Division` (`Ratio`, `FirstFixed`, `SecondFixed`). Sopra l'albero stanno le finestre *flottanti* (`FloatingWindow`: una finestra più un rettangolo, un bordo facoltativo, un titolo e la finestra a cui restituire il focus quando si chiude): il minibuffer, il `*Backtrace*`, i popup dei moduli.



Le foglie si dividono lo spazio; le flottanti sono disegnate sopra, con il loro rettangolo.

Il compartimento `Windows` tiene l'albero, le flottanti, l'id con il focus, il prossimo id, lo stato di un trascinamento del mouse e l'ultimo clic (per il doppio clic entro 400 ms). Le operazioni che tolgono una finestra rispondono con un `Removed` (`No`, `Yes`, `Refocus(id)`) che dice alla facciata se e verso quale finestra spostare il focus.

12.2 snapshot : dalla memoria alla fotografia

`snapshot(&env, larghezza, altezza)` costruisce il `FrameSnapshot`. Prende i lock in un ordine fisso e non fa I/O; il disegno avviene dopo, senza lock.

1. Legge dalle variabili Lisp e dai compartimenti piccoli tutto ciò che serve *prima* di toccare finestre e buffer: `echo-message-timeout`, il tema, l'input in sospeso (`pending_input`), il messaggio della keymap transitoria, `mode-line-format`, `window-separator`, la configurazione del margine (`display-line-numbers`), se la colorazione è in corso.
2. Legge l'area messaggi, vuota se il messaggio è scaduto.
3. Con `windows` in scrittura (lo scorrimento può dover seguire il punto) e `buffers` in lettura, compone ogni foglia dell'albero nel rettangolo del frame meno l'ultima riga, raccogliendo i separatori; poi compone le flottanti.
4. Consuma il testo destinato agli appunti (`take_pending_clipboard`).

Per ogni foglia la composizione fa: scorrimento per mantenere visibile il punto; layout delle righe visibili, compreso il testo virtuale; margine dei numeri di riga (assoluti o relativi, larghezza minima `display-line-numbers-width`, 3 per default, massimo 20); evidenziazioni nell'ordine *sintassi* → *overlay* → *regione* → *testo virtuale*, perché le ultime vincono; posizione del cursore relativa alla finestra; riga di stato. Le finestre flottanti non hanno margine né riga di stato, e ricevono solo regione e testo virtuale (→ TUI-6).

Campo di <code>FrameSnapshot</code>	Contenuto
<code>views</code>	Un <code>RenderableWindowView</code> per finestra: rettangolo, nome del buffer, titolo, focus, cursore relativo, righe di testo, evidenziazioni (riga, colonne, faccia), riga di stato, bordo, margine e sua larghezza.
<code>separators</code>	Rettangoli e carattere delle linee tra finestre affiancate.

Campo di FrameSnapshot	Contenuto
echo_message , prompt	Il messaggio e l'eventuale offerta della keymap transitoria, che lo sostituisce.
pending_input	C-u 4 C-x- : ciò che l'editor sta leggendo.
theme	Gli stili delle facce (Arc<Theme>).
focused_window_id , width , height	Come dice il nome.
colouring_pending	La colorazione non ha finito: ridisegnare tra 40 ms.
clipboard	Testo che il front-end deve mettere negli appunti di sistema, se presente.

La riga di stato segue `mode-line-format` (predefinito " %* %b %m L%l C%c %p "): %b nome del buffer, %f file (o nome), %m modo, %l riga (da 1), %c colonna (da 0), %p posizione (All , Top , Bot o percentuale), %* ** se modificato e -- altrimenti, %%.

12.3 Il renderer della TUI

`render_to` disegna una fotografia in quest'ordine: nasconde il cursore e pulisce lo schermo; disegna i separatori (prima delle finestre, perché una flottante deve poterli coprire); per ogni vista disegna bordo e titolo, ogni riga del rettangolo riempita fino alla larghezza (così una flottante è opaca), il margine cella per cella, le evidenziazioni sopra il testo, la riga di stato sotto il rettangolo; poi l'area messaggi nell'ultima riga, l'input in sospeso allineato a destra nella stessa riga, il cursore *per ultimo* e infine, se c'è, la sequenza OSC 52 per gli appunti.

I colori sono risolti dal front-end secondo la profondità del terminale: 24 bit con `COLORTERM` che contiene `truecolor` o `24bit`; il cubo a 256 colori (o la scala di grigi, se più vicina) con `TERM` che contiene `256color`; altrimenti la voce più vicina dei 16 colori della tavolozza, nell'ordine di `NAMED_COLORS`.

Gli appunti in uscita passano da OSC 52 perché l'editor non sa di avere un terminale e un programma esterno (`xclip`, `pbcopy`) non funzionerebbe via SSH. In entrata non c'è nulla da leggere: il terminale consegna gli appunti con l'incolla «bracketed», che diventa `insert-pasted-text`.

12.4 Il mouse

Con `mouse-mode` attivo la TUI chiede al terminale di riportare il mouse. `handle_mouse_event` traduce l'evento in un `Hit` (testo di una finestra, riga di stato, separatore, flottante, niente), e da lì in un comando: clic per spostare il punto e il focus, doppio clic, trascinamento per selezionare (con scorrimento automatico ogni 60 ms oltre il bordo), trascinamento di un separatore o di una riga di stato per ridimensionare, rotella per scorrere la finestra sotto il puntatore. Risponde `true` solo se ha cambiato qualcosa.

13 Macro di tastiera

Una macro registra *tasti*, non comandi: `C-u 5`, un tasto letto da `read-key-sequence` e perfino un tasto non associato fanno parte di ciò che l'utente ha premuto. `handle_key_event` chiama `record_keys` nei punti in cui un tasto viene consumato. La riproduzione rimanda i tasti a `handle_key_event` dentro un `OneGroup` (un solo gruppo di undo) e si ferma al primo comando fallito, confrontando `command_errors` prima e dopo ogni tasto. Una macro che richiama macro può annidarsi fino a 16 livelli (`MAX_DEPTH`).

14 Le primitive dell'editor

14.1 Il registro

Ogni modulo di `primitives/` ha una funzione `install(into: &Registry)` che registra le sue primitive con `into.function(nome, puntatore, DOC)` o, per i comandi, `into.command(nome, puntatore, specifiche, DOC)`. `install_primitives` chiama tutti i moduli. La macro `primitive!` genera la firma richiesta da `LispPrimitive`; `args.rs` contiene gli aiuti per leggere gli argomenti (nomi come simbolo o stringa, numeri, posizioni).

Ogni docstring comincia con la firma, nella forma `(nome ARG):` seguita da una frase: è ciò che mostrano i comandi di aiuto. `doc_faithfulness_tests` confronta ciò che l'aiuto racconta con l'editor reale; `primitive_shape_tests` verifica le forme condivise dalle primitive, come il controllo dell'arità.

Modulo	N.	Contenuto
<code>edits</code>	47	Inserire, cancellare, uccidere, incollare, undo, movimenti per carattere, parola, riga, paragrafo ed espressione.
<code>io</code>	31	File: aprire, salvare, rileggere, recuperare, rinominare; percorsi e directory; variabili d'ambiente; <code>quit</code> e <code>save-some-buffers</code> .
<code>buffers</code>	17	Creare, cambiare, chiudere buffer; contenuto; sola lettura; dati appesi (<code>buffer-put</code> , <code>buffer-get</code>).
<code>region</code>	17	Mark e regione, <code>keyboard-quit</code> , kill ring (<code>kill-region</code> , <code>yank</code> , <code>yank-pop</code> , <code>kill-new</code>).
<code>modes</code>	15	<code>make-mode</code> , <code>add-hook</code> , regole e regioni di sintassi, <code>add-auto-mode</code> , tabella sintattica, <code>syntax-ppss</code> .
<code>verbs</code>	14	Maiuscole, spazi, righe vuote, unione e duplicazione di righe, trasposizioni, <code>zap-to-char</code> .
<code>macros</code>	12	Registrazione e riproduzione delle macro, contatore.
<code>minibuffer</code>	12	<code>minibuffer-read</code> , conferma, annullamento, completamento, storia.
<code>replace</code>	12	Sostituzione e <code>query-replace</code> .
<code>windows</code>	12	Dividere, chiudere, scegliere, scorrere finestre; <code>display-buffer-at-bottom</code> .
<code>general</code>	11	<code>eval-file</code> , <code>define-key</code> , <code>define-repeat-key</code> , <code>repeat</code> , <code>log</code> , <code>backtrace</code> , <code>regex-opt</code> , keymap transitorie.
<code>results</code>	11	Liste di posizioni (<code>occur</code> , compilazione) e <code>next-error</code> .
<code>commands</code>	10	<code>call-interactively</code> , <code>M-x</code> , <code>register-command</code> e il registro dei comandi.
<code>isearch</code>	10	Ricerca incrementale.
<code>rectangle</code>	10	Operazioni sui rettangoli.
<code>completion</code>	8	<code>completion-at-point</code> , le sue fonti, <code>fuzzy-filter</code> .
<code>match_data</code>	7	<code>string-match</code> e i dati della corrispondenza.
<code>comments</code>	6	Commentare e decommentare.
<code>help</code>	6	Ciò che serve all'aiuto: <code>key-binding</code> , <code>where-is</code> , <code>keymap-bindings</code> , <code>read-key-sequence</code> . I comandi <code>describe-*</code> sono in <code>help.lisp</code> .
<code>mouse</code>	6	Comandi del mouse.
<code>overlays</code>	5	Overlay.
<code>theme</code>	5	Facce e stili.
<code>ask</code>	4	Domande sì/no con keymap transitoria.
<code>shell</code>	4	Comandi di shell.

Modulo	N.	Contenuto
virtual_text	4	Testo virtuale.
workers	4	define-worker, background-call, stop-worker, running-workers.
ui	3	recenter e finestre flottanti.
scan	1	scan-buffer : tutte le corrispondenze di un modello in un buffer, come voci di una lista di risultati.

Il totale è 304, a cui si aggiungono le 86 primitive di base dell'interprete. L'elenco completo dei nomi è nell'appendice (Sezione 18.3).

15 I moduli Lisp

Quasi tutto il comportamento visibile è in `core/lisp/`: le associazioni dei tasti, i comandi di alto livello, i modi dei linguaggi, l'interfaccia dei completamenti. `build.rs` copia i file in `target/<profilo>/data/lisp` a ogni compilazione, da un *elenco esplicito*: un modulo nuovo va aggiunto lì, altrimenti non arriva accanto all'eseguibile e `eval-file` non lo trova.

Modulo	Che cosa aggiunge
<code>commands</code>	<code>defcommand</code> , <code>save-match-data</code> e altre macro di base.
<code>debug</code>	<code>message</code> , <code>report-error</code> , il buffer <code>*Backtrace*</code> , <code>debug-on-error</code> .
<code>common-keymaps</code>	Le associazioni di Emacs: movimento, kill, file, buffer, finestre, <code>C-g</code> .
<code>indent</code>	<code>Tab</code> , <code>indent-region</code> , <code>newline-and-indent</code> , <code>electric-indent-mode</code> .
<code>minibuffer</code>	Tasti e comportamenti del prompt.
<code>rust-mode</code> , <code>risp-mode</code> , <code>cc-mode</code>	Modi per Rust, per il Lisp dell'editor e per C/C++.
<code>dired</code>	Una directory in un buffer (<code>C-x d</code>).
<code>completion</code>	La striscia dei completamenti (<code>*completion-read-function*</code>).
<code>preview</code>	Anteprima di <code>string-rectangle</code> .
<code>clipboard</code>	Copia le uccisioni negli appunti di sistema.
<code>electric-pair</code>	Chiusura automatica di parentesi e virgolette.
<code>buffer-list</code>	<code>C-x b</code> e <code>C-x C-b</code> .
<code>shell</code>	<code>M-!</code> .
<code>manpage</code>	Pagine di manuale.
<code>help</code>	<code>C-h f</code> , <code>C-h k</code> , <code>C-h b</code> , <code>describe-prefix-keys</code> .
<code>occur</code>	<code>M-s o</code> , <code>M-s g</code> .
<code>compile</code>	<code>C-c c</code> e <code>M-g n</code> .
<code>theme</code>	<code>C-c t</code> : scelta del tema.
<code>completion-at-point</code>	Le cinque fonti di <code>C-M-i</code> .

Le variabili che l'editor legge da Lisp:

Variabile	Default	Effetto
<code>echo-message-timeout</code>	5	Secondi di permanenza di un messaggio; <code>nil</code> per sempre.
<code>minibuffer-width</code> , <code>minibuffer-height</code>	60, 3	Dimensioni del prompt.
<code>mode-line-format</code>	vedi sopra	Riga di stato.
<code>window-separator</code>	<code>" "</code>	Carattere tra finestre; <code>nil</code> per uno spazio.
<code>display-line-numbers</code>	<code>nil</code>	<code>t</code> assoluti, <code>'relative</code> relativi.
<code>display-line-numbers-width</code>	3	Cifre minime del margine.
<code>mouse-mode</code>	<code>nil</code> (<code>t</code> in <code>init.lisp</code>)	Il mouse all'editor invece che al terminale.
<code>after-resize-hook</code>	<code>nil</code>	Funzioni chiamate con larghezza e altezza.

Variabile	Default	Effetto
frame-width, frame-height	—	Dimensioni correnti, scritte dal front-end.
lisp-path	data/ lisp	Dove eval-file cerca i moduli.
help-key	"C-h"	Il tasto che, dopo un prefisso, chiede che cosa lo segue.
worker-fuel	200.000	Passi per turno di un worker.
watch-files, watch-file-interval	t, 3	Il watcher dei file.
auto-save, auto-save-interval, auto-save-directory	t, 30, —	L'autosalvataggio.
history-length	100	Voci per storia di prompt.
minibuffer-read- function	default- minibuffer- prompt	Chi apre i prompt.
key-capture- function	nil	Chi riceve la prossima sequenza invece di eseguirla.

16 I test

I test dell'editor stanno in `core/src/tests/`, uno o più file per funzionalità (circa ottanta), più `tests/perf/` per le misure e i test del front-end in `src/tests/`. Costruiscono un editor con `create_global_env`, caricano i moduli Lisp che servono con `include_str!` e valutano Lisp o simulano tasti con `handle_key_event`. `isolate_config_for_tests` punta `RSEDIT_CONFIG_DIR` a una directory temporanea del processo, una volta sola, così i test non scrivono nella configurazione di chi li esegue.

Alcuni file meritano di essere conosciuti prima di cambiare il codice che proteggono: `layering_tests` (i due confini), `deadlock_tests` (i lock attraverso Lisp), `doc_faithfulness_tests` e `primitive_shape_tests` (le docstring), `edit_protection_tests` (sola lettura), `undo_tests`, `frame_snapshot_tests`, `shipped_lisp_tests` (ogni modulo distribuito, caricato nell'ordine di `init.lisp`: nell'editor reale un errore di caricamento finisce solo nel log). `perf::suite` misura tempi reali con più thread e può fallire su macchine lente o condivise (→ [DOC-4](#)).

17 Ricette

17.1 Una primitiva o un comando in Rust

1. Scegli il modulo di `primitives/`; scrivi la docstring come `const`, con la firma nella prima riga.
2. Scrivi la funzione con `primitive!`. Leggi lo stato con le closure della facciata (`with_current_buffer_mut`, `windows`, ...) e *non* chiamare Lisp con un lock preso: copia, rilascia, poi chiama.
3. Modifica il testo solo con `insert_text` e `delete_range`, e controlla il loro risultato.
4. Se scorri qualcosa di grande, addebita il carburante (`ctx.consume_fuel`).
5. Registra con `into.function` o, per un comando, con `into.command` e le specifiche degli argomenti.
6. Aggiungi i test in `core/src/tests/` e aggiorna la tabella di Sezione 14.

17.2 Un comando in Lisp

```
(defcommand count-words-region (start end) ("r")
  "Report how many words the region holds."
  (message "%d words" (length (split-string (buffer-substring start end)))))
(define-key nil "M-=" 'count-words-region)
```

`define-key` con `nil` lega nella keymap globale, con un simbolo di modo nella keymap di quel modo. Un comando legato senza argomenti passa da `call-interactively`, che gli chiede la regione.

17.3 Un modo maggiore

Il modello è `rust-mode.lisp`:

```
(make-mode 'toy-mode)
(add-syntax-region 'toy-mode "\"" "\"\" 'string "\\\".")
(add-syntax-rule 'toy-mode "#.*" 'comment)
(add-syntax-rule 'toy-mode "\\b(let|if|else)\\b" 'keyword)
(set-comment-syntax 'toy-mode '("#"))
(put 'toy-mode 'tab-width 2)
(add-auto-mode "\\..toy$" 'toy-mode)
```

Le regioni prima delle regole; ciò che può attraversare una riga è una regione, il resto una regola. Ricorda che `make-mode` sostituisce un modo esistente.

17.4 Un lavoro di sfondo

In Lisp: `background-call` con una fiber che fa `(yield)` tra un pezzo e l'altro, e una callback `ON-DONE` per il risultato. In Rust: un `ScheduledTask` se il lavoro si fa a turni limitati (copia sotto lock, lavora senza, ricontra la versione prima di salvare), un `ImmediateTask` se può bloccarsi; spediscilo con `send_to_worker`. In entrambi i casi l'effetto sull'interfaccia passa da una callback dovuta.

17.5 Un modulo Lisp nuovo

Scrivi `core/lisp/nome.lisp`, aggiungilo all'elenco di `core/build.rs`, aggiungi `(eval-file "nome")` a `DEFAULT_INIT_LISP` in `editor/boot.rs` (e, a mano, al tuo `init.lisp`), e un test che lo carichi.

17.6 Un nuovo front-end

Un front-end deve: tradurre gli eventi nei tipi di `rsedit_core`; chiamare `handle_key_event`, `handle_paste`, `handle_mouse_event` e `resize`; comunicare `frame-width` e `frame-height`; in un ciclo, chiedere `snapshot` e disegnarlo senza tenere nulla dell'editor; aspettare al massimo `next_redraw_in` (e `drag_scroll_in`) e chiamare `tick` quando l'attesa scade; mettere

`frame.clipboard` negli appunti. Tutto ciò che c'è da sapere su colori, bordi e cursore è nella fotografia;
`src/tui.rs` è l'esempio completo.

18 Appendice

18.1 Glossario

Buffer testo con punto, mark, storia e modo; può visitare un file.

Punto la posizione del cursore in un buffer, in caratteri.

Mark, regione una seconda posizione; la regione è tra mark attivo e punto.

Comando una funzione registrata con le sue specifiche di argomenti.

Keymap transitoria una keymap consultata prima di tutte, finché non viene tolta.

Compartimento un gruppo di campi dello stato sotto un solo lock, in `managers/`.

Facciata `EditorState`, l'unico che tiene più lock di compartimento insieme.

Fotografia il `FrameSnapshot`, ciò che il front-end disegna.

Callback dovuta una chiamata chiesta da un lavoro di sfondo ed eseguita dal thread dei comandi.

Generazione il numero che distingue l'esecuzione viva di un lavoro con nome dalle precedenti.

Epoca il timbro del comando corrente, con cui l'undo raggruppa le modifiche.

18.2 Mantenere aggiornato questo manuale

- Nuovo campo di `EditorState` o nuovo compartimento → Sezione 4 (tabelle e ordine dei lock).
- Cambiamento del percorso di un tasto → Sezione 5; di `call-interactively` → Sezione 6.
- Nuova primitiva o nuovo modulo di `primitives/` → Sezione 14 e l'elenco in appendice (lo script che lo genera cerca `into.function(` e `into.command(`).
- Nuovo lavoro di sfondo → la tabella di Sezione 11.
- Nuovo campo della fotografia o cambiamento del renderer → Sezione 12.
- Nuovo modulo Lisp o nuova variabile letta da Rust → Sezione 15.
- Difetto corretto → aggiorna la descrizione, toglie il rimando al rapporto e la scheda da `rapporto-problemi.typ`.

Gli estratti di codice vanno riletti quando cambia la funzione da cui vengono. I diagrammi usano gli aiuti di `style.typ` descritti nel manuale dell'interprete. Dopo ogni modifica conviene compilare e guardare le pagine.

18.3 Elenco delle primitive dell'editor

Generato da `into.function(` e `into.command(` nei file di `primitives/`. Un asterisco indica un comando, raggiungibile con `M-x`.

Modulo	Primitive
<code>ask</code> (4)	<code>y-or-n</code> , <code>yes-or-no</code> , <code>ask--answer</code> , <code>ask--typed</code>
<code>buffers</code> (17)	<code>buffer-put</code> , <code>buffer-get</code> , <code>buffer-substring</code> , <code>switch-to-buffer</code> , <code>current-buffer</code> , <code>buffer-create</code> , <code>close-buffer</code> , <code>kill-buffer</code> *, <code>kill-buffer-without-saving</code> *, <code>buffer-string</code> , <code>clear-buffer</code> , <code>with-current-buffer</code> , <code>set-buffer-read-only</code> , <code>buffer-read-only-p</code> , <code>buffer-modified-p</code> , <code>buffer-file-name</code> , <code>major-mode</code>
<code>commands</code> (10)	<code>register-command</code> , <code>commandp</code> , <code>command-args</code> , <code>all-commands</code> , <code>execute-extended-command</code> , <code>command-completions</code> , <code>command-execute-prompt</code> *, <code>all-buffer-names</code> , <code>call-interactively</code> , <code>pending-command</code>
<code>comments</code> (6)	<code>comment-dwim</code> *, <code>comment-line</code> *, <code>comment-indent</code> *, <code>comment-region</code> *, <code>uncomment-region</code> *, <code>comment-syntax</code>

Modulo	Primitive
completion (8)	completion-at-point *, fuzzy-filter, completion-at-point-choose, add-completion-function, set-completion-functions, completion-functions, buffer-words, bounds-of-thing-at-point
edits (47)	delete-region, self-insert, insert, insert-pasted-text *, insert-newline *, delete-backward-char *, backward-char *, forward-char *, previous-line *, delete-char *, kill-line *, kill-whole-line *, kill-word *, backward-kill-word *, kill-paragraph *, backward-kill-paragraph *, undo *, redo *, undo-boundary *, set-undo-limit *, beginning-of-line *, end-of-line *, forward-word *, backward-word *, forward-sexp *, backward-sexp *, kill-sexp *, backward-kill-sexp *, up-list *, backward-up-list *, down-list *, current-column, current-indentation, previous-indentation, indent-line-to, forward-paragraph *, backward-paragraph *, beginning-of-buffer *, end-of-buffer *, goto-line *, next-line *, point, point-min, point-max, goto-char, line-number-at-point, current-line
general (11)	eval-file, define-key, define-repeat-key, repeat *, log, regexp-opt, all-logs, backtrace, set-echo-message, set-transient-keymap, clear-transient-keymap
help (6)	key-binding, keys-with-prefix, keymap-bindings, where-is, command-specs, read-key-sequence
io (31)	quit *, quit-without-saving *, save-some-buffers *, save-some-buffers--answer, quit--confirm, find-file *, data-directory, getenv, setenv, path-separator, list-dir, directory-files-recursive, read-file-to-string, match-list, expand-file-name, file-name-as-directory, directory-file-name, file-name-directory, file-name-nondirectory, save-buffer *, save-buffer--overwrite, recover-file *, recover-file--adopt, revert-buffer *, revert-buffer--reread, write-file *, file-exists-p, file-directory-p, directory-entry-count, delete-file, rename-file
isearch (10)	isearch-forward *, isearch-backward *, isearch-forward-regexp *, isearch-backward-regexp *, isearch-update, isearch-exit, isearch-abort, isearch-repeat-forward, isearch-repeat-backward, isearch-active-p
macros (12)	kmacro-start-macro *, kmacro-end-macro *, kmacro-cancel-macro *, kmacro-call-macro *, kmacro-end-and-call-macro *, kmacro-insert-counter *, kmacro-set-counter *, kmacro-counter, kmacro-recording-p, kbd-macro-keys, define-kbd-macro, kmacro-call-keys
match_data (7)	string-match, string-match-p, match-beginning, match-end, match-string, match-data, set-match-data
minibuffer (12)	minibuffer-cleanup, minibuffer-confirm, minibuffer-cancel, minibuffer-complete, minibuffer-choose-completion, default-minibuffer-prompt, minibuffer-read, completion-invalidate, minibuffer-history, clear-minibuffer-history, history-previous *, history-next *

Modulo	Primitive
modes (15)	make-mode, add-hook, add-syntax-rule, add-syntax-region, add-auto-mode, syntax-ppss, balance-point, bounds-of-enclosing-list, set-syntax-pairs, set-string-syntax, set-char-quote, set-syntax-entry, set-comment-syntax, syntax-class, matching-delimiter
mouse (6)	mouse-set-point *, mouse-start-selection *, mouse-drag-to *, mouse-resize *, mouse-mode-toggle *, mouse-scroll *
overlays (5)	make-overlay, delete-overlay, remove-overlays, overlays-at, overlay-face
rectangle (10)	rectangle-mark-mode *, kill-rectangle *, delete-rectangle *, copy-rectangle-as-kill *, yank-rectangle *, open-rectangle *, clear-rectangle *, string-rectangle *, rectangle-bounds, killed-rectangle
region (17)	set-mark *, keyboard-quit *, deactivate-mark *, exchange-point-and-mark *, mark-whole-buffer *, kill-region *, kill-ring-save *, yank *, yank-pop *, set-kill-ring-max *, mark, use-region-p, region-beginning, region-end, kill-new, current-kill, kill-ring-length
replace (12)	query-replace *, query-replace-regexp *, replace-string *, replace-regexp *, replace-this, replace-skip, replace-rest, replace-back, replace-done, replace-abandon, replace-match, replace-count
results (11)	occur--scan, grep--scan, next-error *, previous-error *, results-count, results-entry, results-state, results-visit, results-buffer, results-select *, results-put
scan (1)	scan-buffer
shell (4)	shell-command-start *, shell-command-to-string, parse-overstrike, shell-command-running-p
theme (5)	set-face *, face-style, reset-faces, list-faces, list-colors
ui (3)	recenter *, make-floating-window, close-floating-window
verbs (14)	upcase-word *, downcase-word *, capitalize-word *, delete-horizontal-space *, just-one-space *, delete-blank-lines *, back-to-indentation *, join-line *, duplicate-line *, transpose-lines *, transpose-chars *, transpose-words *, zap-to-char *, zap-to-char--do
virtual_text (4)	make-virtual-text, clear-virtual-text, delete-virtual-text, virtual-text-at
windows (12)	split-window-below *, split-window-right *, delete-window *, delete-other-windows *, other-window *, selected-window, select-window, count-windows, scroll-up-command *, scroll-down-command *, display-buffer-at-bottom, window-buffer
workers (4)	define-worker, stop-worker, running-workers, background-call